A PROJECT REPORT

ON

Design and Implementation of a Real Time Web Based Chat Application
Using MERN Stack and Socket.IO

For the partial fulfillment of the Bachelor of Technology degree from Maulana Abul Kalam Azad University of Technology

In

ELECTRONICS AND COMMUNICATION ENGINEERING

By

Indrajit Bhowmick — 25500322044

Under the Guidance of
Prof. Debasmita Manna
Department of ECE

SurTech

Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex

Affiliated to Maulana Abul Kalam Azad University, West Bengal (Formerly WBUT)

540 Dum Dum Road, Surermath, Kolkata-700074 West Bengal

2025-2026

DECLARATION

I, Indrajit Bhowmick, bearing University Roll Number 25500322044, hereby declare that the project work entitled "Design and Implementation of a Real-Time Web-Based Chat Application Using MERN Stack and Socket.IO" is a genuine and original work carried out by me in the Department of Electronics and Communication Engineering, Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex (Autonomous), 540 Dum Dum Road, Surer Math, Kolkata – 700074, West Bengal.

The project has been completed under proper academic guidance for the partial fulfillment of the Bachelor of Technology degree under Maulana Abul Kalam Azad University of Technology, West Bengal.

Place: Kolkata

Date:

Signature of Student:

DR. SUDHIR CHANDRA SUR INSTITUTE OF TECHNOLOGY & SPORTS COMPLEX
(Formerly known as Dr. Sudhir Chandra Sur Degree Engineering College)
540 Dum Dum Road, Surermath, (Near Dum Dum Jn. Station), Kolkata- 700074
Phone: +91 33 25603889, 25603898, 65330375
Website: www.surtech.edu.in, Email: info@dsec.ac.in

CERTIFICATE OF APPROVAL

This is to certify that Indrajit Bhowmick (Roll No. 25500322044) has submitted the B. Tech project entitled "Design and Implementation of a Real-Time Web-Based Chat Application Using MERN Stack and Socket.IO" for the partial fulfilment of the requirement of the Degree of B. Tech in Electronics and Communication Engineering from Maulana Abul Kalam Azad University (formerly W.B.U.T) in the year 2026.

It is hereby approved and certified as a creditable study of a technological subject carried out and presented in a manner satisfactory towards its acceptance as prerequisite to the Degree of B. Tech in Electronics and Communication Engineering for which it has been submitted.

It is understood that by this approval, the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the report only for the purpose for which it has been submitted.

Signature of the Supervisor

Prof. Debasmita Manna
Assistant Professor
Department of ECE (Electronics and Communication Engineering)
Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex

Signature of the HOD, ECE

Dr. Anirban Neogi
Head of the Department
Department of Electronics and Communication Engineering
Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, support, and encouragement of many individuals. I express my sincere gratitude to Prof. Debasmita Manna, Department of Electronics and Communication Engineering, Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex, for her valuable guidance, constant supervision, and technical support throughout the development of this project.

I would also like to thank Dr. Anirban Neogi, Head of the Department, Electronics and Communication Engineering, for providing me with the necessary facilities and academic support required for the successful completion of this work. I am equally thankful to all faculty members and staff of the department for their encouragement and cooperation.

Finally, I express my heartfelt gratitude to my friends and family members for their continuous motivation and support throughout the completion of this project.

Student Name with University Roll Number and Signature :-

INDRAJIT BHOWMICK (25500322044) : _____

ABSTRACT

Real-time communication has become an essential requirement in modern web applications, enabling users to exchange information instantly across different locations. Traditional web applications primarily rely on the HTTP request-response communication model, which is not well suited for applications requiring continuous interaction due to increased latency and repeated client requests. Modern communication systems overcome these limitations by employing persistent bidirectional communication protocols that support efficient and low-latency message exchange.

Various real-time messaging platforms have been developed using advanced communication technologies and distributed architectures. Although these platforms provide rich features such as instant messaging, multimedia sharing, and secure authentication, many of them are proprietary systems whose internal implementation is not accessible for educational purposes. Consequently, there is a need for a practical and modular implementation that demonstrates the fundamental concepts of real-time web application development using modern open-source technologies.

This project presents the Design and Implementation of a Real-Time Web-Based Chat Application Using MERN Stack and Socket.IO. The proposed system follows a client-server architecture in which React.js is used to develop the frontend user interface, Node.js and Express.js manage backend services, MongoDB stores user and message data, and Socket.IO enables real-time bidirectional communication. The application incorporates secure user authentication using JSON Web Tokens (JWT) and bcrypt, profile image management through Cloudinary, and global state management using Zustand to provide a responsive and seamless user experience.

The developed application supports user registration, secure login, profile management, one-to-one real-time messaging, online user status, and persistent conversation storage. The modular architecture ensures efficient communication, simplified maintenance, and scalability for future enhancements. Functional testing confirmed the successful implementation of all major system features, demonstrating stable performance, secure authentication, and low-latency message delivery under normal operating conditions.

In conclusion, the proposed real-time web-based chat application successfully demonstrates the practical implementation of modern full-stack web development technologies for building secure and responsive communication systems. Although the current implementation focuses on one-to-one messaging, the modular design provides a strong foundation for future extensions such as group chats, voice and video communication, end-to-end encryption, AI-assisted messaging, and cloud-based deployment.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1 : INTRODUCTION

1.1 Introduction

Communication is one of the most essential aspects of human interaction, enabling individuals and organizations to exchange information efficiently regardless of geographical boundaries. Over the past few decades, advancements in networking technologies, cloud computing, and web application development have transformed communication from traditional methods such as letters, telephone calls, and emails into highly interactive digital platforms capable of delivering information almost instantaneously. Today, real-time communication systems form the backbone of numerous applications including social networking platforms, online education, customer support services, collaborative workspaces, healthcare systems, financial services, and enterprise communication solutions.

Modern users expect communication platforms to provide immediate message delivery, real-time notifications, seamless synchronization across devices, and an intuitive user experience. Applications such as instant messaging systems have therefore become an indispensable part of everyday life. Unlike conventional web applications that rely solely on the HTTP request-response model, modern communication platforms require continuous bidirectional communication between clients and servers to ensure that messages are transmitted and received instantly without requiring users to manually refresh the application.

The rapid growth of full-stack JavaScript technologies has significantly simplified the development of such applications. The MERN technology stack, consisting of **MongoDB**, **Express.js**, **React.js**, and **Node.js**, has emerged as one of the most popular frameworks for developing scalable web applications. By allowing JavaScript to be used across the frontend, backend, and database interactions, the MERN stack provides a unified development environment that improves productivity, maintainability, and scalability. Additionally, technologies such as **Socket.IO** enable event-driven communication over persistent WebSocket connections, making real-time messaging practical, efficient, and reliable.

The project titled **"Design and Implementation of a Real-Time Web-Based Chat Application Using MERN Stack and Socket.IO"** focuses on developing a secure and responsive communication platform capable of supporting authenticated users, instant one-to-one messaging, online user presence detection, profile management, and cloud-based media sharing. The application follows a modular client-server architecture in which the frontend is responsible for user interaction, the backend manages business logic and API services, MongoDB stores application data, and Socket.IO facilitates real-time communication between connected users.

Apart from implementing the core functionality of a messaging platform, this project serves as a comprehensive study of modern web development principles. During its development, several important concepts such as RESTful API design, client-server communication, authentication and authorization, database management, event-driven programming, state management, responsive user interface development, and cloud service integration have been explored and implemented in a practical environment.

The project demonstrates how multiple technologies can work together to build an efficient and scalable communication system while maintaining high levels of performance, security, and usability.

Furthermore, the architecture of the proposed system has been designed with extensibility in mind. Future enhancements such as group conversations, end-to-end encryption, voice and video calling, push notifications, AI-powered assistants, multilingual message translation, intelligent search mechanisms, and advanced administrative features can be incorporated without requiring significant modifications to the existing system architecture. This modular approach ensures that the application remains maintainable and adaptable to evolving user requirements and technological advancements.

The successful implementation of this project demonstrates the practical application of contemporary software engineering principles and modern web technologies in developing a real-time communication platform. It also provides valuable hands-on experience in full-stack application development and establishes a strong foundation for future research and development in the field of web-based communication systems.

Layer	**Technology**	**Purpose**
Frontend	React.js	User Interface Development
Backend	Node.js & Express.js	Server-side Application Logic
Database	MongoDB	Data Storage
Real-Time Communication	Socket.IO	Instant Message Exchange
Authentication	JWT	Secure User Authentication
Password Security	bcrypt	Password Hashing
Media Storage	Cloudinary	Profile Image Storage
State Management	Zustand	Global State Management
Styling	Tailwind CSS & DaisyUI	Responsive User Interface

Table 1.1 Major Technologies Used

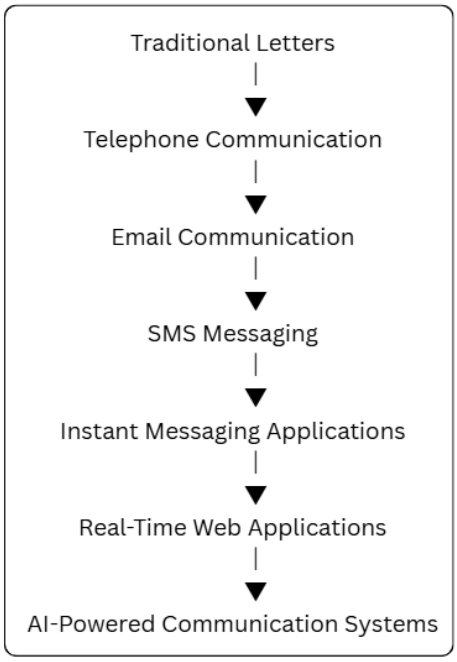


Figure 1.1: Evolution of digital communication technologies from conventional methods to modern real-time communication systems.

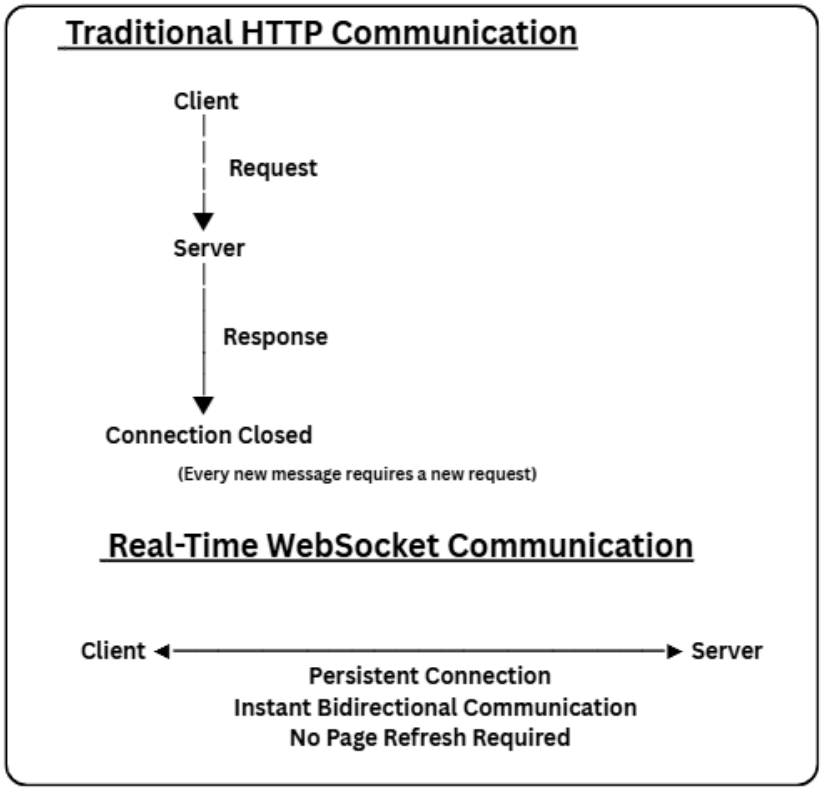


Figure 1.2: Comparison between the traditional HTTP request-response model and persistent WebSocket communication used in the proposed real-time chat application.

1.2 Background

The rapid advancement of information and communication technologies has fundamentally transformed the way individuals, organizations, and businesses interact with one another. In today's digital era, communication is no longer limited by geographical boundaries or time constraints. The widespread availability of high-speed internet, smartphones, cloud computing, and modern web technologies has enabled users to communicate instantly from virtually anywhere in the world. As a result, real-time communication systems have become an essential component of everyday life, supporting applications ranging from personal messaging and social networking to online education, healthcare, customer support, and enterprise collaboration.

In the early stages of web development, most web applications were designed using the traditional client-server architecture based on the Hypertext Transfer Protocol (HTTP). Under this communication model, the client sends a request to the server, and the server processes the request before returning a response. Once the response is delivered, the connection is terminated. While this approach is highly effective for static websites and conventional business applications, it is not suitable for applications that require continuous synchronization and instant data exchange between multiple users. In such systems, frequent polling or repeated page refreshes become necessary, leading to increased server load, higher network traffic, and delayed communication.

The increasing demand for instant messaging applications has driven the development of technologies capable of supporting persistent and bidirectional communication between clients and servers. WebSocket technology was introduced to address these limitations by establishing a continuous communication channel that remains active throughout a user's session. Unlike traditional HTTP communication, WebSockets allow both the client and server to exchange data at any time without repeatedly establishing new connections. This significantly reduces communication latency while improving the responsiveness and overall user experience of web applications.
Modern web development has also witnessed the emergence of powerful full-stack frameworks that simplify the development of scalable and maintainable applications. Among these, the MERN stack has become one of the most widely adopted technology stacks for full-stack JavaScript development. The MERN stack consists of MongoDB, Express.js, React.js, and Node.js, each performing a specific role within the application architecture. React.js provides a dynamic and interactive frontend user interface, Express.js and Node.js handle server-side processing and API development, while MongoDB offers a flexible NoSQL database for efficient storage and retrieval of application data. Since all components are based on JavaScript, developers can build complete web applications using a single programming language across the entire software stack.

Another significant advancement in real-time application development is the introduction of Socket.IO, a JavaScript library built on top of the WebSocket protocol. Socket.IO simplifies the implementation of event-driven communication by providing automatic reconnection, connection management, broadcasting, room-based communication, and fallback mechanisms for browsers that do not support native WebSockets. These capabilities make Socket.IO particularly suitable for developing applications such as chat systems, live notification platforms, collaborative editing

tools, multiplayer games, and monitoring dashboards where real-time synchronization is critical.

Security has also become a major concern in modern web applications due to the increasing amount of sensitive user information being processed online. Authentication mechanisms such as JSON Web Tokens (JWT) enable secure user verification by allowing authenticated users to access protected resources without repeatedly transmitting credentials. Similarly, password hashing algorithms such as bcrypt enhance application security by storing encrypted passwords instead of plain text, thereby reducing the risk of unauthorized access even if the database is compromised.

The proposed project, **"Design and Implementation of a Real-Time Web-Based Chat Application Using MERN Stack and Socket.IO,"** has been developed by integrating these modern technologies into a unified communication platform. The application combines secure authentication, responsive user interface design, cloud-based media management, database persistence, RESTful APIs, and event-driven communication to provide users with a seamless real-time messaging experience. Beyond serving as a communication platform, the project demonstrates the practical implementation of modern software engineering principles, scalable system architecture, and full-stack web development practices.

Overall, the background presented in this section establishes the technological foundation upon which the proposed system has been designed and implemented. It also highlights the evolution of communication technologies and the necessity of adopting modern frameworks and real-time communication protocols to meet the growing demands of today's digital society.

Technology	**Purpose**	**Role in the Proposed System**
React.js	Frontend Development	Interactive User Interface
Node.js	Runtime Environment	Server-side Execution
Express.js	Web Framework	REST API Development
MongoDB	NoSQL Database	Storage of User and Message Data
Socket.IO	Real-Time Communication	Instant Message Exchange
JWT	Authentication	Secure User Login and Session Management
Cloudinary	Cloud Storage	Profile Image Management

Table 1.2 Major Technologies Supporting Real-Time Communication

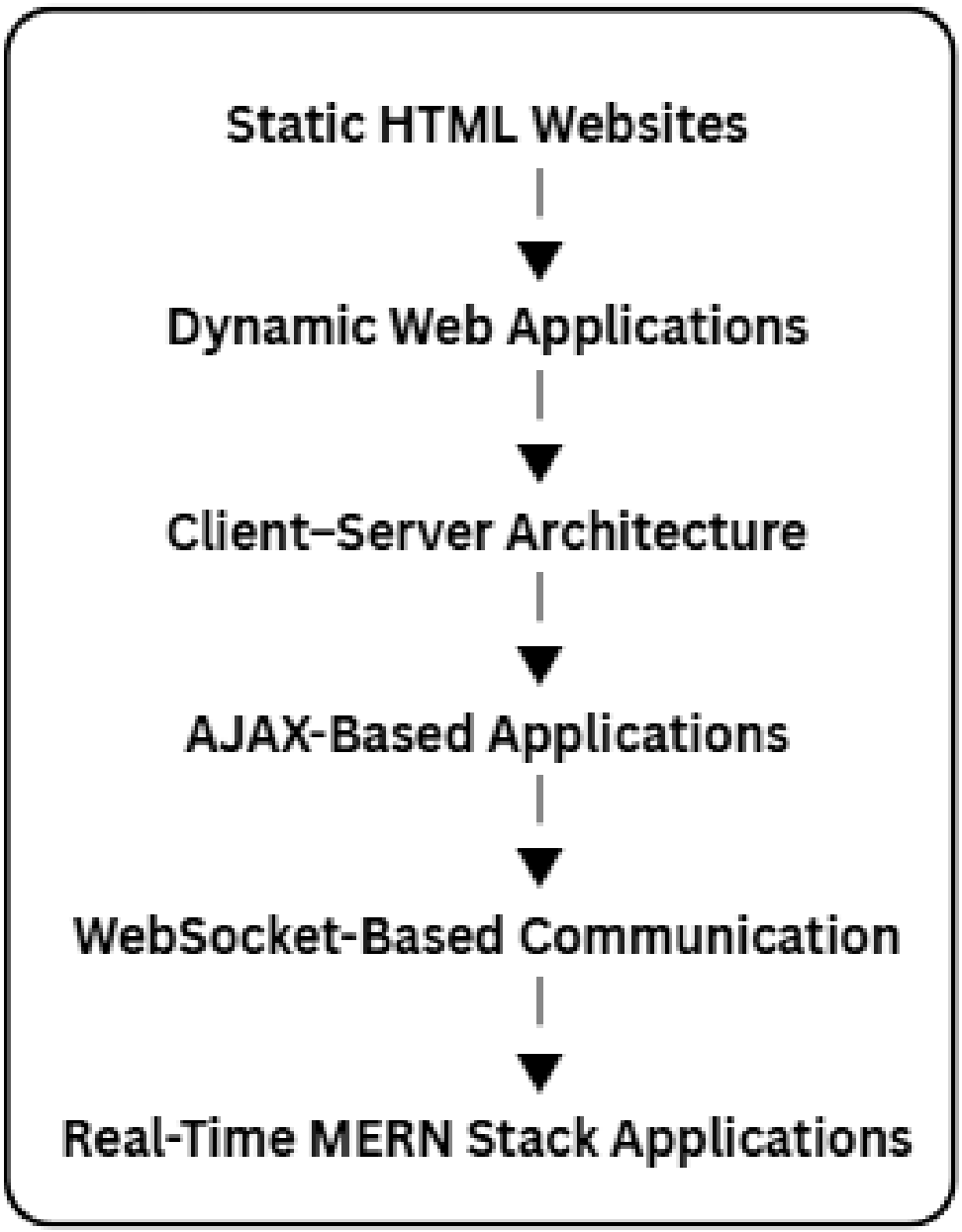


Figure 1.3: Evolution of web application technologies leading to modern real-time

1.3 Motivation

The increasing dependence on digital communication has transformed the way individuals interact in both personal and professional environments. With the rapid growth of remote working, online education, social networking, and virtual collaboration, users expect communication systems to provide instant message delivery, high reliability, and seamless user experiences. Modern messaging applications have become an essential part of daily life, enabling people to communicate across geographical boundaries in real time. Consequently, there is a growing demand for web-based communication platforms that are secure, scalable, responsive, and capable of supporting real-time interactions.

Traditional web applications primarily rely on the HTTP request-response communication model, where a client sends a request to the server and waits for a response before initiating another interaction. Although this approach is suitable for conventional web applications, it is inefficient for real-time communication systems because users must repeatedly refresh the application or rely on periodic polling to receive updated information. Such mechanisms increase server workload, consume additional network resources, and introduce noticeable delays in message delivery, thereby reducing the overall user experience.

Existing messaging platforms such as WhatsApp, Telegram, Discord, and Slack provide rich communication features and high scalability. However, these platforms are proprietary systems with complex infrastructures that are not easily accessible for academic study or software development practice. Their internal architectures, communication protocols, and implementation strategies are generally not available for detailed analysis. As a result, students often gain experience only as end users rather than understanding the engineering principles involved in designing and implementing a modern real-time communication system.

The primary motivation behind this project was to gain practical experience in designing and developing a complete full-stack web application capable of supporting secure real-time communication. Rather than studying individual technologies in isolation, this project provides an opportunity to integrate multiple modern development frameworks into a single cohesive system. Throughout the development process, practical knowledge has been acquired in frontend development, backend development, database management, RESTful API design, authentication mechanisms, cloud service integration, state management, and event-driven communication.

Another important motivation was to understand how persistent communication protocols such as WebSockets differ from traditional HTTP communication. Implementing Socket.IO enabled the development of an application where messages are transmitted instantly between connected users without requiring manual page refreshes. This provided valuable insight into event-driven architectures, socket connection management, message broadcasting, user presence detection, and synchronization between multiple clients connected to the server simultaneously.

Security considerations also served as a significant motivation during the development of the project. Modern web applications frequently process sensitive user information, making secure authentication and authorization essential. To address

this requirement, the proposed system incorporates JSON Web Token (JWT)-based authentication, password encryption using bcrypt, secure API access, and protected application routes. These mechanisms ensure that only authenticated users can access system resources while improving the overall security of the application.

The project was further motivated by the need to develop a modular and scalable application architecture capable of supporting future enhancements. Instead of designing the application solely for its current functionality, the system architecture has been organized into independent frontend and backend modules with clearly defined responsibilities. Such an approach improves maintainability, simplifies future development, and allows new features to be incorporated with minimal modifications to the existing codebase. Potential future enhancements include group conversations, voice and video calling, message search, AI-powered virtual assistants, multilingual translation, end-to-end encryption, push notifications, and advanced administrative controls.

Finally, this project represents an opportunity to apply theoretical concepts learned throughout the undergraduate curriculum in a practical software engineering environment. Concepts such as client-server architecture, database systems, networking, operating systems, web technologies, software engineering principles, and information security have been integrated into a real-world application. The successful implementation of this project not only demonstrates technical proficiency in full-stack web development but also strengthens problem-solving skills, system design capabilities, and understanding of scalable software architecture.

Overall, the motivation behind this project extends beyond the development of a simple messaging application. It reflects an effort to understand the design principles, architectural patterns, and implementation techniques involved in building modern real-time communication systems while creating a strong foundation for future research and development in web application engineering.

Motivation	**Expected Outcome**
Learn full-stack development	Practical implementation of MERN Stack
Understand real-time communication	Implementation of Socket.IO and WebSockets
Improve application security	Secure authentication using JWT and bcrypt
Gain software engineering experience	Development of a modular and scalable application
Apply theoretical knowledge	Integration of networking, databases, and web technologies

Table 1.3 Motivation and Expected Outcomes

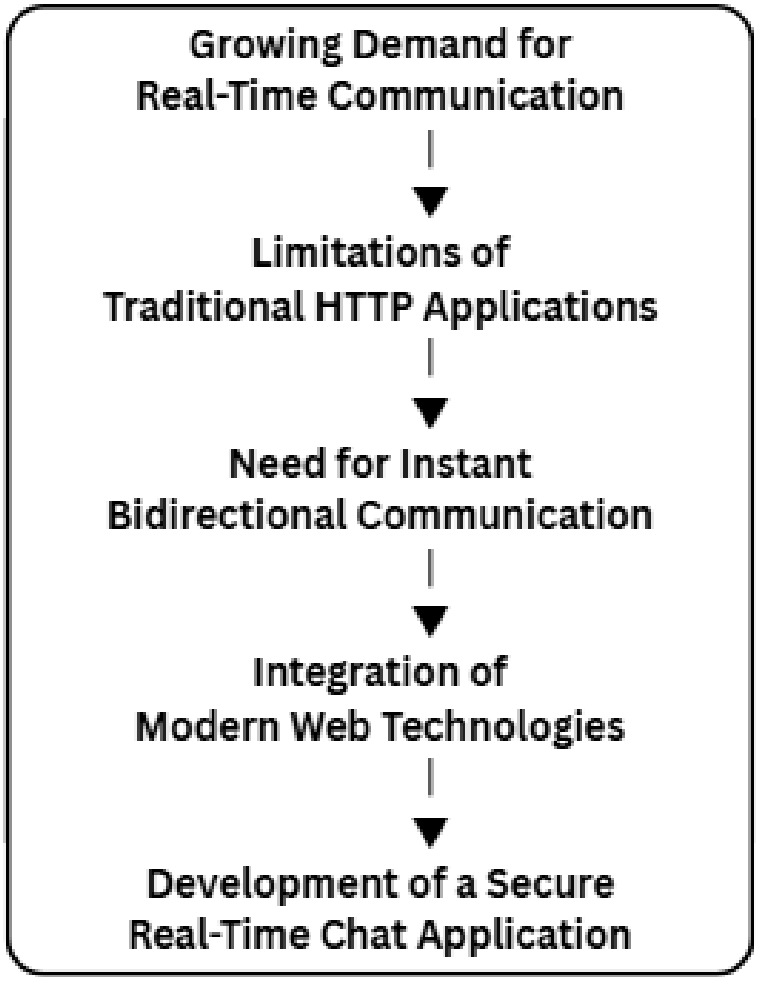


Figure 1.4: Factors that motivated the development of the proposed real-time web-based chat application.

1.4 Problem Statement

The rapid growth of internet-based communication has significantly increased the demand for applications capable of delivering instant, reliable, and secure interactions between users. Traditional web applications primarily operate using the HTTP request-response communication model, where a client sends a request to the server and receives a corresponding response before the connection is terminated. Although this communication model is suitable for static websites and conventional web applications, it is not well suited for systems requiring continuous synchronization and real-time information exchange. In applications such as online messaging platforms, collaborative workspaces, and customer support systems, users expect messages and updates to appear instantly without manually refreshing the web page.

Conventional HTTP-based communication introduces several limitations when applied to real-time messaging systems. Since each interaction requires a separate request and response cycle, continuous polling is often necessary to detect new messages. This approach increases server workload, consumes additional network bandwidth, and introduces unnecessary communication delays. As the number of active users increases, the repeated polling mechanism becomes increasingly inefficient, resulting in poor scalability and reduced application performance. These limitations negatively affect the overall user experience by preventing truly real-time communication.

Another significant challenge involves maintaining synchronized communication between multiple users connected simultaneously. In a real-time chat environment, every message sent by one user must be delivered immediately to the intended recipient while simultaneously updating the user interface without interrupting the ongoing session. Achieving this level of synchronization requires a persistent communication mechanism capable of supporting bidirectional data exchange between the client and server. Traditional web communication methods are unable to efficiently support these requirements without additional technologies or complex implementation strategies.

Security also represents a major concern in modern communication platforms. Chat applications process sensitive information such as user credentials, personal details, profile images, and private conversations. Without proper authentication and authorization mechanisms, unauthorized users may gain access to protected resources, leading to potential privacy violations and security risks. Therefore, secure user authentication, encrypted password storage, protected application routes, and session validation are essential requirements for any modern messaging application.

Many existing messaging platforms provide advanced communication features, but they are generally proprietary systems with closed architectures and highly complex infrastructures. Their implementation details are not openly available for academic learning or software engineering practice. Consequently, there is a need for a lightweight, modular, and educational real-time communication system that demonstrates the practical implementation of modern web technologies while maintaining security, scalability, and maintainability.

The proposed project addresses these challenges by designing and implementing a real-time web-based chat application using the MERN stack and Socket.IO. The

application replaces traditional polling mechanisms with persistent WebSocket-based communication to enable instantaneous message delivery. Secure user authentication is achieved using JSON Web Tokens (JWT), password security is enhanced through bcrypt hashing, and MongoDB provides flexible and scalable data storage for user information and chat messages. The resulting system demonstrates an efficient and practical solution for modern web-based communication while providing a strong foundation for future feature enhancements and scalability.

Existing Problem	**Proposed Solution**
Delayed message delivery	Real-time communication using Socket.IO
Repeated HTTP requests	Persistent WebSocket connection
High server load due to polling	Event-driven communication model
Insecure authentication methods	JWT-based authentication with protected routes
Plain-text password storage	Password hashing using bcrypt
Limited scalability	Modular MERN stack architecture
Poor user experience	Responsive React-based interface with instant updates

Table 1.4 Existing Problems and Proposed Solutions

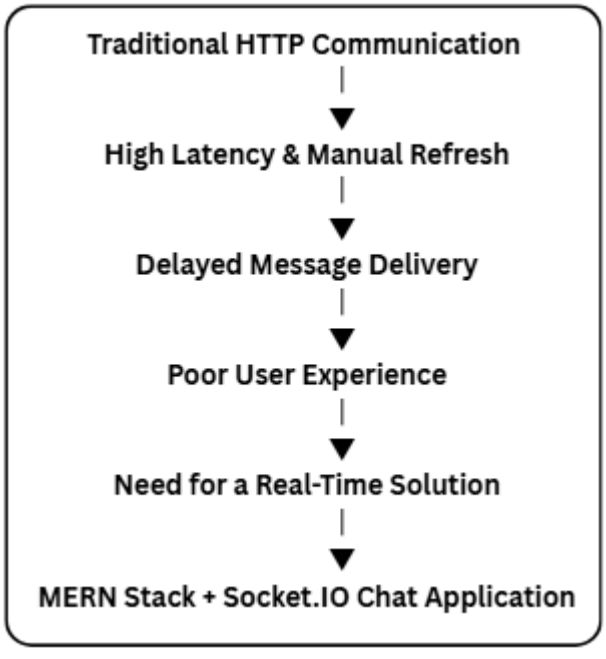


Figure 1.5: Identification of the problem addressed by the proposed real-time chat application.

1.5 Proposed Solution

To overcome the limitations associated with conventional web-based communication systems, this project proposes the design and implementation of a **Real-Time Web-Based Chat Application** developed using the **MERN Stack** and **Socket.IO**. The proposed system provides a secure, scalable, and responsive communication platform capable of supporting instant one-to-one messaging through persistent bidirectional communication between clients and the server.

Unlike traditional messaging systems that depend solely on the HTTP request-response model, the proposed application integrates WebSocket-based communication using Socket.IO. This enables the establishment of a persistent connection between the client and the server, allowing messages to be transmitted and received instantly without requiring manual page refreshes or repeated polling requests. As a result, users experience seamless real-time communication with significantly reduced latency and improved responsiveness.

The application follows a modular **client-server architecture**, where each layer of the system performs a specific responsibility. The frontend is developed using **React.js**, providing an interactive, responsive, and component-based user interface. React's efficient rendering mechanism enables dynamic updates of chat messages, user information, and online status without reloading the application.

The backend is implemented using **Node.js** and **Express.js**, which together provide a robust environment for handling RESTful APIs, authentication, business logic, and communication with the database. The server manages incoming client requests, validates user information, processes messaging operations, and coordinates real-time communication through Socket.IO.

The application utilizes **MongoDB**, a document-oriented NoSQL database, to store user accounts, profile information, and chat messages. Its flexible document structure allows efficient storage and retrieval of dynamic chat data while supporting future scalability. The interaction between the application and the database is managed using **Mongoose**, which provides schema-based data modeling and simplifies database operations.

Security is incorporated as one of the fundamental design principles of the proposed system. User authentication is implemented using **JSON Web Tokens (JWT)**, ensuring that only authenticated users can access protected resources and communicate with other users. Passwords are securely stored using the **bcrypt** hashing algorithm, preventing exposure of plain-text credentials even in the event of unauthorized database access. Protected API routes further enhance system security by validating user identity before processing requests.

To improve the user experience, the application also supports profile image management through **Cloudinary**, a cloud-based media storage service. Instead of storing image files directly within the server or database, profile images are uploaded to Cloudinary, which returns optimized URLs that are stored in the database. This approach reduces server storage requirements, improves image delivery performance, and simplifies media management.

Application state management is handled using **Zustand**, which provides lightweight and efficient global state management for user information, authentication status, selected conversations, and application settings. Combined with **Axios** for HTTP communication and **React Router** for navigation, the frontend provides a smooth and responsive user experience across different pages of the application.

The complete system has been designed following software engineering principles that emphasize modularity, maintainability, and scalability. Each major functionality, including authentication, messaging, profile management, database interaction, and socket communication, is implemented as an independent module. This modular architecture simplifies future maintenance and enables additional features such as group chats, file sharing, voice and video communication, AI-powered assistants, and end-to-end encryption to be integrated with minimal modifications to the existing codebase.

Overall, the proposed solution demonstrates how modern web technologies can be effectively combined to develop a secure, efficient, and scalable real-time communication platform. The integration of the MERN stack with Socket.IO provides an ideal foundation for developing interactive web applications that require continuous synchronization and instant data exchange while maintaining high standards of security, reliability, and performance.

Component	**Technology Used**	**Primary Responsibility**
User Interface	React.js	Displays application screens and manages user interaction
Backend Server	Node.js & Express.js	Processes requests, business logic, and API handling
Database	MongoDB	Stores users, messages, and application data
Database ODM	Mongoose	Schema modeling and database interaction
Real-Time Engine	Socket.IO	Instant message transmission and online user tracking
Authentication	JWT	User authentication and authorization
Password Security	bcrypt	Password hashing and secure credential storage
Cloud Storage	Cloudinary	Profile image upload and management
State Management	Zustand	Global application state management
API Communication	Axios	Client-server communication using REST APIs

Table 1.5 Major Components of the Proposed System

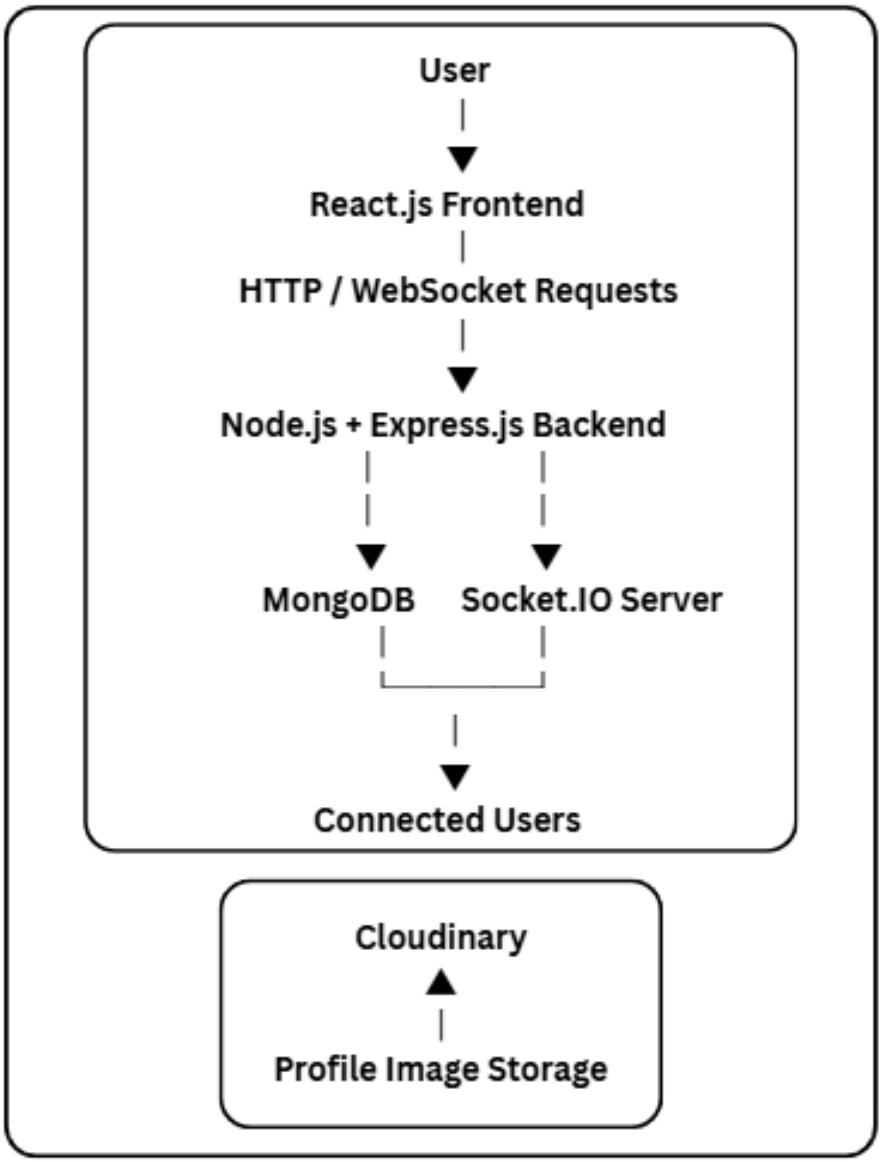


Figure 1.6: High-level architecture of the proposed real-time web-based chat application.

CHAPTER 2 : LITERATURE SURVEY

Chapter Overview

This chapter presents a comprehensive review of the technologies, communication protocols, and existing messaging platforms related to the proposed real-time web-based chat application. It examines the evolution of digital communication systems, discusses the underlying technologies that enable real-time messaging, reviews widely used communication applications, and compares their features and limitations. Finally, the chapter identifies the research gap that motivates the development of the proposed system.

2.1 Introduction

Communication technologies have undergone significant transformation over the past few decades, evolving from traditional text-based communication methods to sophisticated real-time messaging platforms capable of supporting millions of concurrent users. The rapid growth of internet infrastructure, cloud computing, and modern web technologies has enabled communication systems to become faster, more reliable, and highly interactive. Today, real-time communication plays a crucial role in numerous domains, including education, healthcare, business collaboration, customer support, social networking, and remote work environments.

Modern users expect messaging applications to provide instant message delivery, low latency, secure communication, media sharing, and seamless synchronization across multiple devices. These expectations have encouraged researchers and software developers to design communication systems that are scalable, efficient, and capable of handling continuous interactions between users. As a result, technologies such as WebSockets, event-driven architectures, cloud databases, and full-stack JavaScript frameworks have become fundamental components of modern communication platforms.

The development of web-based chat applications has progressed significantly with the introduction of technologies that enable persistent client-server communication. Traditional web applications relied heavily on the HTTP request-response communication model, which was not designed for continuous real-time interaction. To overcome these limitations, communication protocols such as WebSocket and libraries such as Socket.IO were introduced, enabling full-duplex communication channels between clients and servers. These technologies have revolutionized the development of messaging applications by providing instant data transmission with minimal latency.

Several commercial messaging platforms have successfully implemented these technologies to provide reliable communication services. Applications such as WhatsApp, Telegram, Discord, and Slack demonstrate how modern software architectures can support millions of active users while maintaining high availability, scalability, and security. Although these applications offer rich functionality, their proprietary implementations limit opportunities for academic study and practical

understanding of the underlying software engineering concepts.
In addition to communication protocols, advancements in full-stack development frameworks have significantly simplified application development. The MERN stack has emerged as one of the most widely adopted technology stacks for developing modern web applications due to its unified JavaScript ecosystem, modular architecture, and flexibility. Combined with cloud-based services, secure authentication mechanisms, and responsive frontend frameworks, the MERN stack provides an ideal foundation for building scalable real-time communication systems.

This chapter reviews the major technologies and communication systems relevant to the proposed project. It discusses the evolution of communication technologies, examines the principles of real-time communication, compares existing messaging platforms, and analyzes their strengths and limitations. The insights obtained from this literature survey provide the foundation for identifying the research gap addressed by the proposed real-time web-based chat application.

Generation	**Technology**	**Major Characteristics**
First	Postal Letters	Physical communication, slow delivery
Second	Telephone	Voice-based communication
Third	Email	Internet-based asynchronous messaging
Fourth	SMS & Instant Messaging	Faster text communication
Fifth	Real-Time Web Applications	Instant bidirectional communication
Sixth	AI-Powered Communication	Intelligent automation and personalization

Table 2.1 Evolution of Digital Communication

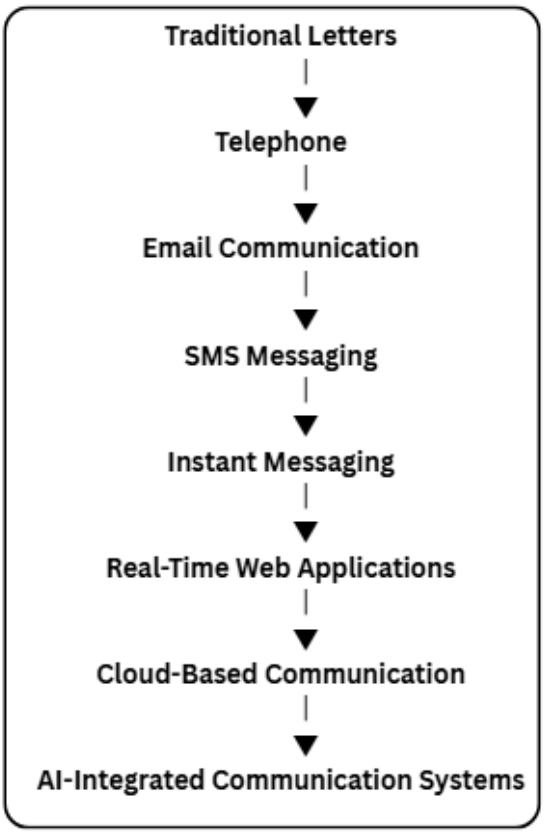


Figure 2.1: Evolution of communication technologies from conventional methods to intelligent real-time communication platforms.

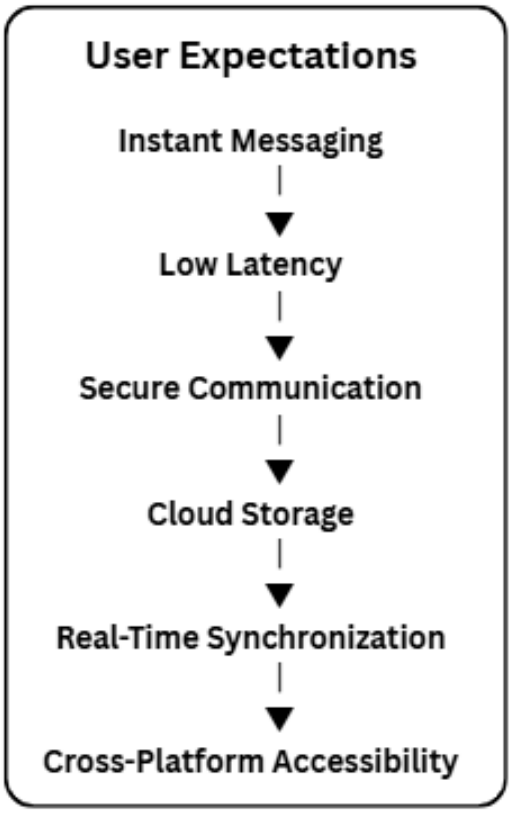


Figure 2.2: Major requirements of modern communication platforms.

2.2 Evolution of Real-Time Communication Systems

The evolution of communication systems has been driven by the continuous demand for faster, more reliable, and more interactive methods of exchanging information. From traditional postal services to modern cloud-based messaging platforms, communication technologies have undergone significant advancements to meet the increasing expectations of users. Each stage of this evolution introduced new technologies that improved communication speed, accessibility, scalability, and user experience.

Initially, communication relied on physical methods such as handwritten letters and printed documents. Although these methods provided reliable communication over long distances, they were extremely slow and unsuitable for situations requiring immediate information exchange. The invention of the telephone revolutionized communication by enabling real-time voice conversations, reducing communication delays from days to minutes. However, voice communication lacked the ability to store conversations efficiently and required both participants to be available simultaneously.

The widespread adoption of the Internet during the late twentieth century introduced electronic communication methods such as electronic mail (Email). Email enabled users to exchange digital messages quickly while supporting attachments, document sharing, and communication across different geographical regions. Despite these advantages, email remained an asynchronous communication medium because recipients typically read messages after they had been delivered rather than participating in continuous conversations.

The growing popularity of mobile devices led to the emergence of Short Message Service (SMS), allowing users to exchange text messages almost instantly. Although SMS significantly improved communication speed compared to email, it suffered from limitations such as message length restrictions, dependency on telecommunication networks, and limited support for multimedia content.

The next major advancement came with the development of internet-based instant messaging applications. Platforms such as MSN Messenger, Yahoo Messenger, Skype, and later WhatsApp, Telegram, and Discord transformed digital communication by enabling continuous conversations over the Internet. These applications introduced several important features including presence detection, multimedia sharing, typing indicators, message synchronization, and group communication. Unlike earlier communication methods, instant messaging applications emphasized low-latency communication and interactive user experiences.

Initially, many web-based messaging applications relied on techniques such as periodic polling and long polling to simulate real-time communication. In these approaches, client applications repeatedly sent requests to the server to check for new messages. While functional, these methods generated unnecessary network traffic, increased server workload, and introduced delays between message transmission and reception. As the number of concurrent users increased, these approaches became increasingly inefficient and difficult to scale.

To overcome these limitations, the WebSocket protocol was introduced as a

standardized solution for full-duplex communication between web browsers and servers. Unlike traditional HTTP communication, WebSocket establishes a persistent connection that remains active throughout the communication session. Once the connection has been established, both the client and server can exchange information independently without repeatedly initiating new requests. This significantly reduces communication latency while improving network efficiency and overall application responsiveness.

Modern communication platforms further simplify WebSocket implementation through libraries such as Socket.IO. Socket.IO provides an event-driven communication framework capable of automatically handling reconnections, connection failures, browser compatibility issues, broadcasting mechanisms, and room-based communication. These capabilities have made Socket.IO one of the most widely adopted technologies for developing real-time applications including messaging platforms, collaborative editing systems, multiplayer games, online learning platforms, and live monitoring dashboards.

Simultaneously, cloud computing and full-stack JavaScript development have accelerated the growth of scalable communication systems. Modern technology stacks such as MERN allow developers to build complete web applications using a unified JavaScript ecosystem, simplifying development while improving maintainability. Combined with cloud storage services, secure authentication mechanisms, and responsive frontend frameworks, these technologies have established a strong foundation for the next generation of real-time communication systems.

Today, communication systems continue to evolve with the integration of Artificial Intelligence, cloud-native architectures, end-to-end encryption, predictive messaging, intelligent assistants, multilingual translation, and advanced collaboration tools. These innovations continue to improve communication efficiency while expanding the capabilities of modern messaging platforms.

The proposed project builds upon these technological advancements by integrating modern web development technologies, persistent real-time communication, secure authentication mechanisms, and cloud-based services into a unified web application capable of delivering an efficient and scalable messaging experience.

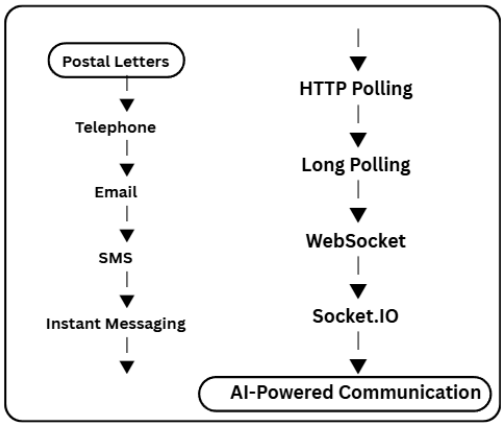


Figure 2.3: Evolution of communication technologies leading to modern real-time messaging systems.

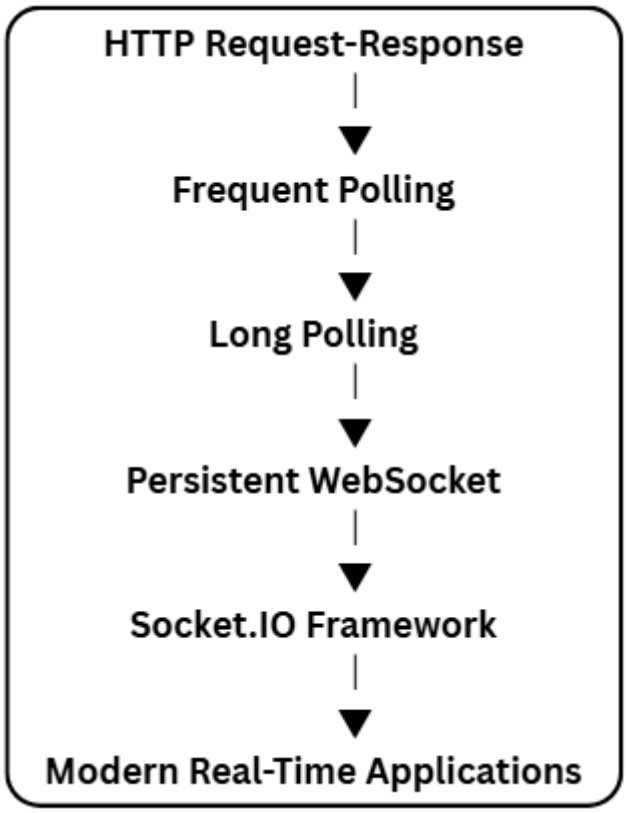


Figure 2.4: Evolution of web communication techniques from conventional HTTP communication to persistent real-time communication.

Communication Method	Communication Type	Speed	Real-Time Support	Major Limitation
Postal Letter	Physical	Very Slow	No	Long delivery time
Telephone	Voice	Fast	Yes	No message persistence
Email	Digital	Moderate	No	Asynchronous communication
SMS	Mobile	Fast	Limited	Character limitations
HTTP Polling	Web	Moderate	Partial	High server overhead
Long Polling	Web	Better	Partial	Increased resource usage
WebSocket	Web	Very Fast	Yes	Requires persistent
Socket.IO	Web	Very Fast	Yes	Additional library dependency

Table 2.2 Comparison of Communication Technologies

Era	Technology	Major Advancement
Pre-Internet	Postal Services	Long-distance written communication
Early Internet	Email	Digital messaging
Mobile Era	SMS	Instant text communication
Web Era	HTTP-Based Chat	Browser-based messaging
Modern Era	WebSocket & Socket.IO	Real-time bidirectional communication
Future Era	AI-Based Communication	Intelligent and context-aware messaging

Table 2.3 Milestones in the Evolution of Communication

2.3 Review of Existing Communication Technologies

Modern real-time web applications are built upon several communication technologies that enable efficient data exchange between clients and servers. Over the years, different communication techniques have been developed to improve response time, reduce network overhead, and provide seamless user experiences. Each technology has its own advantages and limitations depending on the nature of the application. Understanding these communication mechanisms is essential for selecting the most appropriate architecture for a real-time chat application.

This section reviews the major communication technologies used in modern web applications, including the traditional Hypertext Transfer Protocol (HTTP), RESTful APIs, WebSocket protocol, and the Socket.IO framework. Their characteristics, working principles, strengths, and limitations are discussed to justify the selection of Socket.IO as the primary communication mechanism in the proposed system.

2.3.1 Hypertext Transfer Protocol (HTTP)

The **Hypertext Transfer Protocol (HTTP)** is the fundamental communication protocol used by the World Wide Web. It follows a **request-response** communication model, where the client initiates a request and the server processes the request before returning an appropriate response. After the response is delivered, the connection is terminated.

HTTP is highly efficient for retrieving web pages, images, documents, and other static resources. It is also widely used for implementing RESTful APIs that perform operations such as user authentication, database access, and resource management.

However, HTTP was not originally designed for continuous real-time communication. Since every interaction requires a new request and response cycle, applications that depend entirely on HTTP often experience increased latency when delivering frequent updates. Real-time messaging systems built solely on HTTP generally require repeated polling mechanisms to check for new information, resulting in unnecessary network traffic and increased server workload.

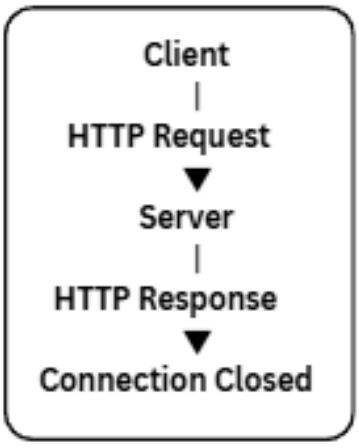


Figure 2.5: Basic request-response communication model using HTTP.

Advantages of HTTP

- Simple and standardized communication protocol.
- Universally supported by web browsers.
- Suitable for RESTful API development.
- Stateless architecture simplifies scalability.
- Efficient for static and transactional applications.
-

Limitations of HTTP

- Not suitable for continuous real-time communication.
- Requires repeated requests for updated information.
- Higher latency compared to persistent communication protocols.
- Increased server workload under frequent polling.

2.3.2 RESTful APIs

Representational State Transfer (REST) is one of the most widely adopted architectural styles for designing web services. RESTful APIs allow clients and servers to exchange information using standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

Within modern web applications, REST APIs are commonly used for operations that involve database interaction or application management rather than continuous communication. Examples include user registration, login, profile updates, retrieving user information, and loading historical chat messages.
In the proposed chat application, REST APIs are responsible for handling authentication, user management, and database operations, while Socket.IO manages real-time message transmission.

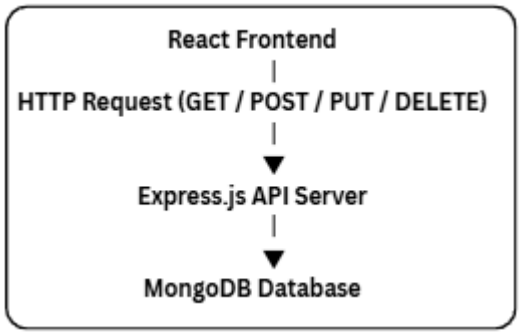


Figure 2.6: General workflow of RESTful API communication.

Advantages of REST APIs

- Standardized architecture.
- Platform independent.
- Easy integration with frontend frameworks.
- Supports CRUD operations efficiently.

- Highly scalable.

Limitations

- Stateless communication.
- Inefficient for instant updates.
- Requires multiple requests for continuous synchronization.

2.3.3 WebSocket Protocol

The **WebSocket protocol** was introduced to overcome the limitations of HTTP in real-time applications. Unlike HTTP, WebSocket establishes a **persistent full-duplex communication channel** between the client and the server. Once the initial handshake is completed, both parties can exchange data independently without repeatedly opening new connections.

This persistent connection significantly reduces latency and network overhead, making WebSocket an ideal solution for chat applications, multiplayer games, financial trading systems, collaborative editing platforms, and live monitoring systems.

Because messages are transmitted immediately after they are generated, WebSocket enables users to experience truly real-time communication.

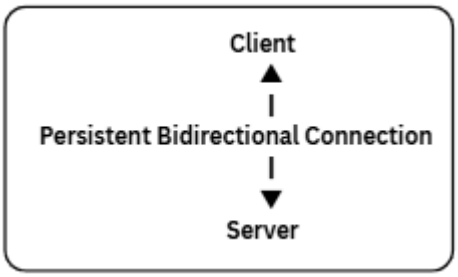


Figure 2.7: Persistent bidirectional communication established through the WebSocket protocol.

Advantages

- Very low communication latency.
- Full-duplex communication.
- Persistent connection.
- Reduced network overhead.
- Ideal for real-time applications.

Limitations

- Requires connection management.
- Native implementation can become complex.
- Does not automatically handle reconnections.

2.3.4 Socket.IO

Socket.IO is a JavaScript library built on top of the WebSocket protocol that simplifies the development of real-time applications. It provides an event-driven programming model while automatically handling reconnection, fallback mechanisms, browser compatibility, room management, broadcasting, acknowledgements, and connection lifecycle management.

Unlike native WebSocket implementations, Socket.IO abstracts many low-level networking complexities, allowing developers to focus on application logic rather than communication management.

The proposed chat application utilizes Socket.IO to establish persistent communication channels between users. Whenever a user sends a message, Socket.IO immediately delivers the event to the server, which forwards it to the intended recipient without requiring additional HTTP requests.

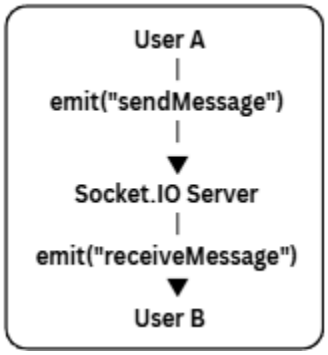


Figure 2.8: Event-driven message transmission using Socket.IO.

Advantages

- Built on top of WebSocket.
- Automatic reconnection.
- Event-driven architecture.
- Supports rooms and namespaces.
- Browser compatibility.
- Simplifies real-time programming.
-

Limitations

- Slightly larger overhead than native WebSocket.
- Requires Socket.IO libraries on both client and server.

Feature	HTTP	REST API	WebSocket	Socket.IO
Communication Type	Request – Response	Request – Response	Full Duplex	Event Driven
Persistent Connection	✗	✗	✓	✓
Real-Time Communication	✗	✗	✓	✓
Low Latency	✗	✗	✓	✓
Automatic Reconnection	✗	✗	✗	✓
Event-Based Messaging	✗	✗	✗	✓
Suitable for Chat Applications	Poor	Poor	Good	Excellent

Table 2.4 Comparison of Communication Technologies

Technology	Primary Purpose
HTTP	Initial webpage loading
REST APIs	Authentication and CRUD operations
WebSocket	Persistent communication protocol
Socket.IO	Real-time messaging and event handling

Table 2.5 Technologies Used in the Proposed System

Section Summary

The review of communication technologies demonstrates that although HTTP and REST APIs remain essential for conventional web applications, they are insufficient for applications requiring continuous real-time interaction. WebSocket overcomes these limitations by providing persistent bidirectional communication, while Socket.IO further enhances this capability through event-driven communication, automatic reconnection, and simplified implementation. These advantages make Socket.IO the most appropriate communication technology for the proposed real-time web-based chat application.

2.4 Review of Existing Messaging Applications

The rapid advancement of internet technologies has led to the development of numerous messaging platforms that facilitate real-time communication between users. These applications provide various features such as instant messaging, multimedia sharing, voice and video communication, online presence detection, cloud synchronization, and secure authentication. They have significantly improved digital communication by enabling users to exchange information efficiently across different devices and geographical locations.

Although these applications have become highly successful, each platform has been developed with different objectives, architectural designs, and feature sets. Some platforms primarily focus on personal communication, while others emphasize business collaboration, team productivity, or community interaction. Studying these applications provides valuable insights into the design principles, technologies, and software architectures adopted in modern communication systems.

This section reviews several widely used messaging applications that have influenced the design of the proposed real-time web-based chat application.

2.4.1 WhatsApp

WhatsApp is one of the most widely used instant messaging platforms, serving billions of users worldwide. The application provides real-time messaging, voice and video calling, multimedia sharing, status updates, location sharing, and end-to-end encrypted communication.

The platform emphasizes simplicity, reliability, and security while supporting communication across multiple devices. Messages are synchronized almost instantly, providing users with a seamless communication experience.

From a software engineering perspective, WhatsApp demonstrates the importance of scalable server architecture, efficient message synchronization, secure authentication, and low-latency communication. However, its proprietary architecture limits opportunities for academic study and software implementation.

Major Features

- Real-time messaging
- End-to-end encryption
- Voice and video calls
- Multimedia sharing
- Group conversations
- Cross-platform synchronization

Limitations

- Closed-source implementation
- Limited customization
- Proprietary communication protocols

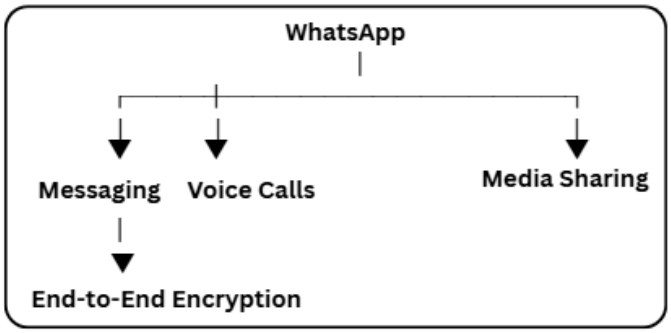


Figure 2.9: Core functionalities provided by WhatsApp.

2.4.2 Telegram

Telegram is a cloud-based messaging platform designed to provide high-speed communication, cloud synchronization, and support for large communities. Unlike many traditional messaging applications, Telegram stores user data in the cloud, allowing conversations to remain synchronized across multiple devices.

Telegram supports channels, bots, file sharing, cloud storage, and API integration, making it suitable for both personal communication and large-scale information broadcasting.

The application demonstrates how cloud computing can improve accessibility, synchronization, and scalability while supporting advanced automation features.

Major Features

- Cloud synchronization
- Large file sharing
- Bots and automation
- Group communication
- Public channels
- Multi-device support
-

Limitations

- End-to-end encryption is optional
- More complex user interface than traditional messaging applications

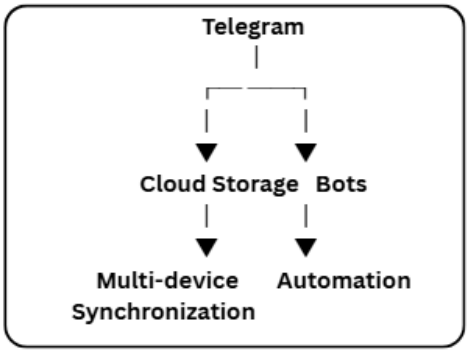


Figure 2.10: Major architectural capabilities supported by Telegram.

2.4.3 Discord

Discord is a communication platform originally developed for gaming communities but has evolved into a comprehensive collaboration platform. Unlike traditional messaging applications, Discord combines text communication, voice channels, video conferencing, and community management within a single platform.

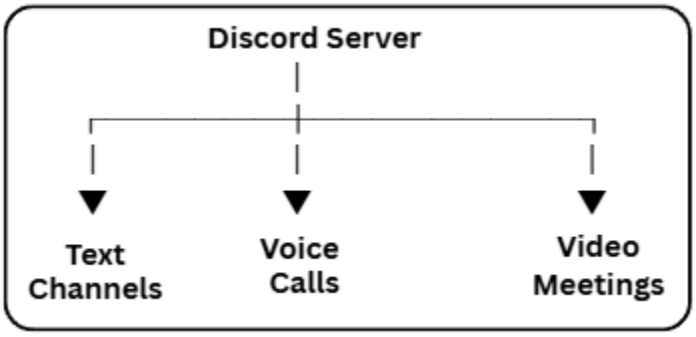
Discord demonstrates the advantages of event-driven communication architectures capable of supporting thousands of simultaneous users within organized communities.

Major Features

- Community servers
- Voice communication
- Video conferencing
- Screen sharing
- Text channels
- Role-based permissions

Limitations

- Higher resource consumption
- Interface complexity for new users
- Designed primarily for communities rather than simple messaging



.

Figure 2.11: Communication services provided by Discord.

2.4.4 Slack

Slack is a cloud-based communication platform primarily designed for professional collaboration and enterprise environments. Unlike personal messaging applications, Slack organizes communication into channels dedicated to projects, departments, or teams.

The platform integrates numerous third-party services including GitHub, Google Drive, Jira, Zoom, and Microsoft Office, making it an effective productivity tool for organizations.

Major Features

- Team collaboration
- Project channels
- File sharing
- Application integrations
- Search functionality
- Workflow automation

Limitations

- Premium features require paid subscriptions
- Designed primarily for business communication
- Complex configuration for small organizations

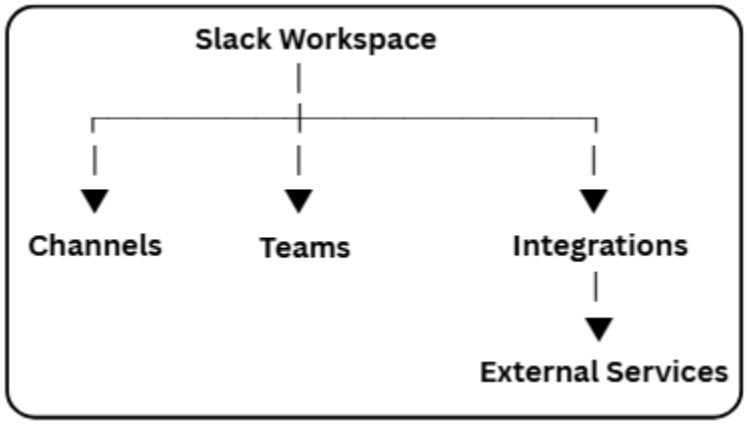


Figure 2.12: Organizational communication model used by Slack.

Feature	WhatsApp	Telegram	Discord	Slack
Real-Time Messaging	✓	✓	✓	✓
Voice Calls	✓	✓	✓	Limited
Video Calls	✓	✓	✓	✓
Group Communication	✓	✓	✓	✓
Cloud Synchronization	Partial	✓	✓	✓
Bot	Limite	✓	✓	✓

Feature	WhatsApp	Telegram	Discord	Slack
Support	d			
Enterprise Collaboration	✗	Limited	Limited	✓
Open Source	✗	Partial	✗	✗

Table 2.6 Comparison of Existing Messaging Applications

Application	Major Strength	Major Limitation
WhatsApp	Simple and secure communication	Closed-source architecture
Telegram	Cloud synchronization and automation	Optional end-to-end encryption
Discord	Community and multimedia communication	Resource-intensive
Slack	Enterprise collaboration	Subscription-based premium features

Table 2.7 Strengths and Limitations of Existing Platforms

Discussion

The review of existing messaging applications indicates that modern communication platforms successfully implement real-time messaging using scalable architectures and persistent communication mechanisms. Each platform has been developed to address different communication requirements, ranging from personal messaging to enterprise collaboration.

Despite their rich functionality, these platforms are proprietary systems with closed implementations that limit opportunities for academic understanding and software engineering experimentation. Furthermore, their extensive infrastructures introduce complexities that are beyond the scope of undergraduate software development projects.

These observations highlight the need for a lightweight, modular, and educational real-time communication platform that demonstrates the practical implementation of modern web technologies while remaining simple enough for academic study and future enhancement. The proposed chat application has therefore been designed using widely adopted open-source technologies including the MERN stack and Socket.IO, providing a practical implementation of the concepts discussed throughout this literature survey.

2.5 Comparative Analysis of Existing Systems

The study of existing messaging platforms reveals that modern communication systems have significantly improved the efficiency and reliability of digital interactions through the adoption of real-time communication technologies, cloud infrastructure, and scalable software architectures. Applications such as WhatsApp, Telegram, Discord, and Slack have successfully demonstrated the advantages of instant messaging, multimedia sharing, and cross-platform accessibility. However, each platform has been developed to satisfy different communication requirements and therefore possesses its own strengths and limitations.

WhatsApp primarily focuses on personal communication by offering secure messaging, voice and video calls, and end-to-end encryption through a simple and user-friendly interface. Telegram emphasizes cloud-based synchronization, support for large communities, and extensive bot integration. Discord is designed around community-based communication with organized servers, voice channels, and collaborative interactions, whereas Slack primarily targets enterprise collaboration through workspace management and third-party service integration.

Although these platforms provide advanced communication capabilities, they are implemented as proprietary systems whose internal architectures, communication protocols, and source code are not publicly available for academic study. Furthermore, many of their advanced features introduce additional complexity that is unnecessary for understanding the fundamental principles of real-time web application development.

The proposed chat application focuses on implementing the core concepts behind modern real-time communication systems using open-source technologies. Rather than replicating every feature available in commercial platforms, the proposed system demonstrates secure authentication, persistent real-time messaging, responsive user interface design, cloud-based media management, and modular software architecture within a simplified educational framework. This approach provides a clear understanding of the software engineering concepts involved while maintaining flexibility for future enhancements.

Feature	WhatsApp	Telegram	Discord	Slack	Proposed System
Real-Time Messaging	✓	✓	✓	✓	✓
User Authentication	✓	✓	✓	✓	✓
Responsive Web Interface	Limited	✓	✓	✓	✓
One-to-One Chat	✓	✓	✓	✓	✓
Group Chat	✓	✓	✓	✓	Future Work
Voice Calling	✓	✓	✓	Limited	Future Work
Video Calling	✓	✓	✓	✓	Future Work
Profile Management	✓	✓	✓	✓	✓
Image Sharing	✓	✓	✓	✓	✓
Open-Source Learning	✗	Partial	✗	✗	✓
Educational Implementation	✗	✗	✗	✗	✓

Table 2.8: Comparative analysis of existing messaging platforms and the proposed real-time web-based chat application.

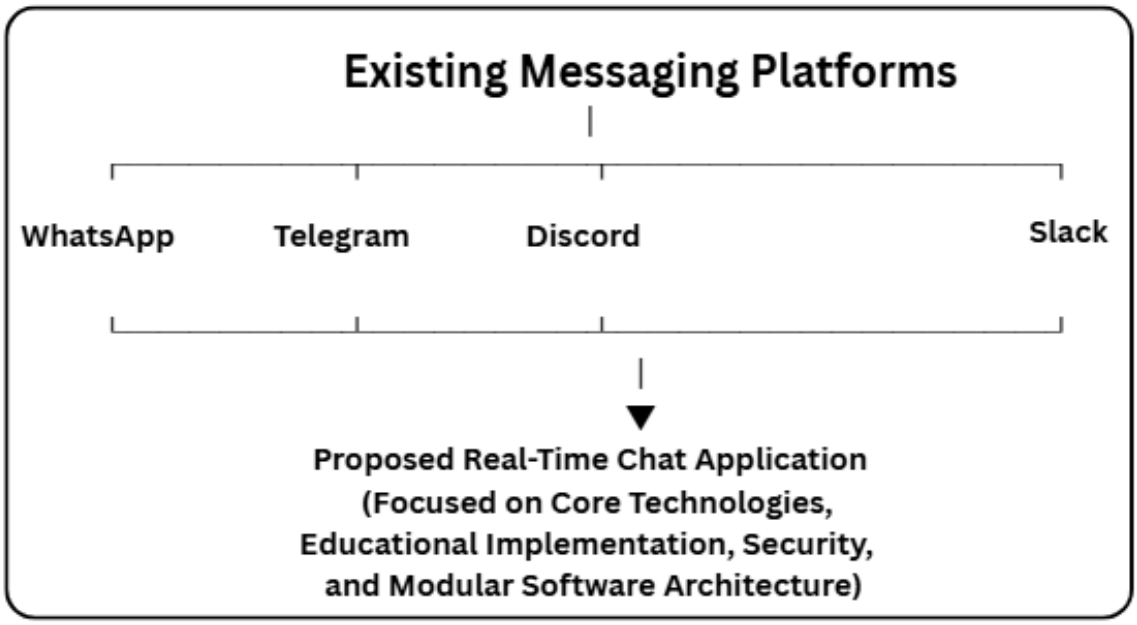


Figure 2.13: High-level comparison between existing messaging platforms and the proposed system.

2.6 Research Gap

The literature survey indicates that existing communication technologies and messaging platforms have significantly advanced real-time digital communication by providing features such as instant messaging, multimedia sharing, cloud synchronization, and secure authentication. Commercial applications including WhatsApp, Telegram, Discord, and Slack have demonstrated the effectiveness of modern communication architectures in supporting millions of users worldwide. However, these platforms are primarily developed as proprietary products with complex infrastructures that are optimized for commercial deployment rather than academic learning or software engineering research.

Although these applications provide extensive functionality, their internal implementation details, architectural decisions, communication workflows, and source code are generally inaccessible for detailed analysis. Consequently, students and developers often understand the features offered by these applications but have limited opportunities to study or implement the underlying technologies responsible for real-time communication.

Furthermore, many existing platforms incorporate numerous advanced features such as enterprise collaboration tools, voice and video communication, workflow automation, and large-scale cloud services. While these capabilities enhance user experience, they also increase architectural complexity and make it difficult to focus on the fundamental concepts involved in developing a real-time messaging system.
The review of communication technologies also reveals that traditional HTTP-based communication is not suitable for applications requiring continuous data synchronization and instant message delivery. Although WebSocket and Socket.IO effectively overcome these limitations, there remains a need for a practical implementation that integrates these technologies with modern full-stack web development frameworks in a simple, modular, and educational manner.

The proposed project addresses this gap by designing and implementing a real-time web-based chat application using the MERN stack and Socket.IO. The system combines secure authentication, responsive user interface development, persistent bidirectional communication, cloud-based media management, and modular software architecture into a unified application. Unlike commercial messaging platforms, the proposed system is developed with an educational perspective, allowing the practical implementation of modern web technologies while maintaining simplicity, scalability, and ease of understanding. It also establishes a strong foundation for future enhancements such as group communication, multimedia sharing, voice and video calling, artificial intelligence integration, and advanced security mechanisms.

Observation	Existing Systems	Proposed System
Source Code Availability	Mostly Closed Source	Fully Developer Controlled
Educational Understanding	Limited	High
Implementation Simplicity	Complex	Modular and Easy to Understand
Real-Time Communication	Available	Implemented using Socket.IO
Scalability	High	Designed for Future Expansion
Learning Opportunity	Limited	Practical Full-Stack Implementation

Table 2.9: Comparison highlighting the research gap and the contribution of the proposed system.

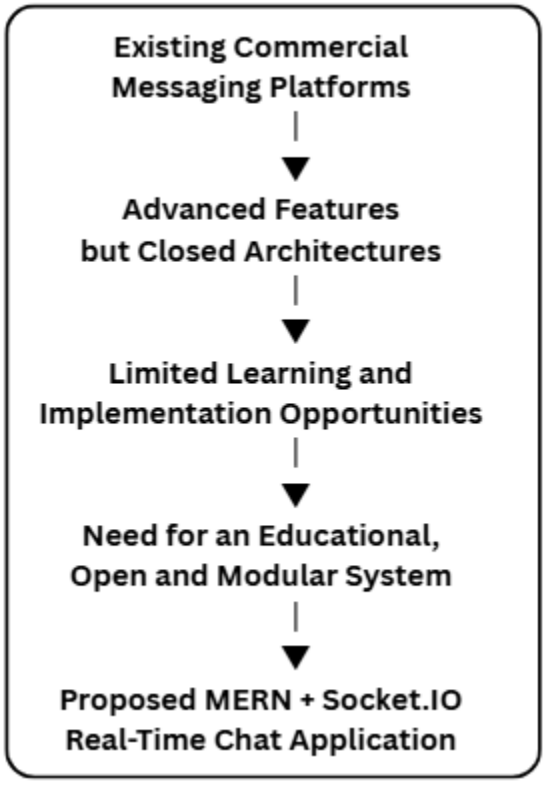


Figure 2.14: Identification of the research gap addressed by the proposed system.

2.7 Chapter Summary

This chapter presented a comprehensive review of the communication technologies, messaging platforms, and software concepts relevant to the development of the proposed real-time web-based chat application. The discussion began with the evolution of digital communication systems, illustrating how communication technologies have progressed from traditional methods to modern real-time web applications capable of supporting instant and interactive communication.

The chapter further examined the fundamental communication technologies that enable modern web applications, including HTTP, RESTful APIs, WebSocket, and Socket.IO. Their working principles, advantages, and limitations were analyzed to demonstrate the significance of persistent bidirectional communication in real-time applications. The study established that while HTTP and REST APIs remain essential for conventional web services, technologies such as WebSocket and Socket.IO provide the low-latency communication required by modern messaging systems.

A review of widely used messaging applications, including WhatsApp, Telegram, Discord, and Slack, was also presented. Their major features, architectural approaches, strengths, and limitations were analyzed to understand current industry practices in real-time communication. The comparative analysis highlighted that although these platforms successfully provide advanced communication services, their proprietary implementations and complex architectures limit their suitability for academic learning and practical software engineering studies.

Based on the literature review, a research gap was identified, emphasizing the need for a modular, educational, and open implementation of a real-time communication platform using modern web technologies. To address this gap, the proposed system integrates the MERN stack with Socket.IO to develop a secure, scalable, and responsive web-based chat application that demonstrates the practical application of full-stack development, real-time communication, and modern software engineering principles.

The knowledge and observations presented in this chapter provide the theoretical foundation for the subsequent chapters. The next chapter focuses on the analysis and design of the proposed system, including system architecture, functional requirements, database design, software modules, and the overall workflow adopted during the development of the application.

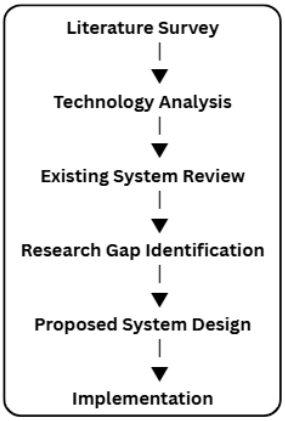


Figure 2.15: Transition from the literature survey to the design and implementation of the proposed system.

CHAPTER 3 : SYSTEM ANALYSIS AND DESIGN

Chapter Overview

This chapter presents the analysis and design of the proposed real-time web-based chat application. It describes the limitations of traditional messaging systems, introduces the proposed solution, and explains the overall system architecture, database design, software requirements, and functional modules. The chapter also illustrates how different components of the application interact to provide secure and efficient real-time communication.

3.1 Existing System

Before designing the proposed chat application, it is important to understand the limitations of traditional web-based communication systems. Conventional messaging applications developed using only the HTTP request-response model require the client to send repeated requests to the server in order to receive updated information. This approach, commonly known as polling, increases network traffic, consumes additional server resources, and introduces delays in message delivery.

Many existing communication platforms successfully provide real-time messaging; however, they are generally designed as proprietary commercial systems with complex infrastructures and closed implementations. Their source code, communication architecture, and deployment strategies are not publicly available, making them unsuitable for understanding the practical implementation of real-time communication technologies from a software engineering perspective.

Furthermore, traditional web applications often lack persistent communication channels between clients and servers. Every request establishes a new connection, resulting in higher latency and inefficient resource utilization for applications that require continuous interaction.

Another limitation is scalability. As the number of users increases, repeated HTTP requests generate unnecessary server load, affecting overall system performance. Applications developed without proper modular architecture also become difficult to maintain, extend, and integrate with new features.

These limitations highlight the need for a modern communication system capable of supporting persistent bidirectional communication, secure authentication, modular software architecture, and efficient data management.

Limitation	Description
Communication Delay	Messages are not delivered instantly due to repeated request-response cycles.
Frequent Polling	Continuous polling increases server workload and network traffic.
Lack of Persistent	Every interaction requires establishing a new

Limitation	Description
Connection	HTTP connection.
Scalability Issues	System performance degrades as the number of users increases.
Limited Educational Value	Proprietary implementations restrict understanding of internal architectures.

Table 3.1 Limitations of Existing Systems

Figure 3.1: Traditional request-response communication used by conventional web applications.

3.2 System Architecture

The system architecture defines the overall structure of the proposed real-time web-based chat application and illustrates how different components interact to provide secure, efficient, and real-time communication. The application follows a **client-server architecture**, where the frontend, backend, database, and cloud storage work together to process user requests and deliver instant messaging services.

The frontend of the application is developed using **React.js**, which provides an interactive and responsive user interface. User actions such as registration, login, profile updates, and message transmission are initiated from the frontend and communicated to the backend through REST APIs and Socket.IO connections.

The backend is implemented using **Node.js** and **Express.js**, which handle authentication, request processing, business logic, and communication with the database. REST APIs are responsible for operations such as user registration, login, profile management, and message retrieval, while Socket.IO establishes persistent communication channels for real-time messaging.

MongoDB serves as the primary database for storing user information, authentication details, and chat messages. Mongoose is used to model the database structure and simplify interactions with MongoDB. Profile images are stored in **Cloudinary**, while only their corresponding URLs are maintained in the database, reducing server storage requirements and improving scalability.

The integration of these technologies forms a modular architecture where each component performs a specific responsibility while interacting seamlessly with the others. This separation of concerns improves maintainability, scalability, and simplifies future enhancements.

Layer	Technology	Purpose
Frontend	React.js	User Interface Development
Styling	Tailwind CSS, DaisyUI	Responsive UI Design
Backend	Node.js, Express.js	Server-side Logic
Database	MongoDB, Mongoose	Data Storage and Management
Authentication	JWT, bcrypt	User Authentication and Password Security
Real-Time Communication	Socket.IO	Instant Message Exchange
Media Storage	Cloudinary	Profile Image Storage
State Management	Zustand	Global State Management

Table 3.5 Technology Stack

Component	Description
User Interface	Allows users to interact with the application.
Authentication Module	Handles registration, login, and user verification.
Chat Module	Manages message transmission and reception.
Database Module	Stores user and message data.
Media Module	Uploads and manages profile images.
Socket Server	Maintains real-time communication between users.

Table 3.6 Major System Components

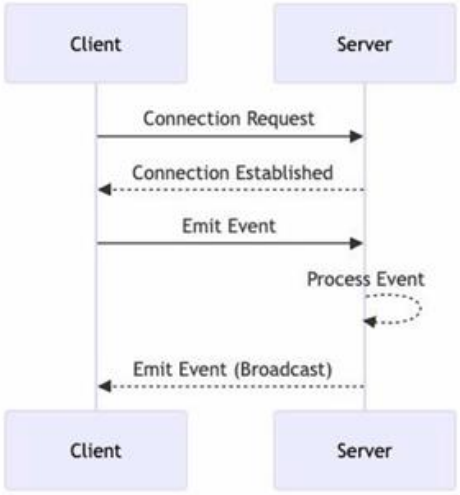


Figure 3.3: Overall architecture of the proposed real-time web-based chat application.

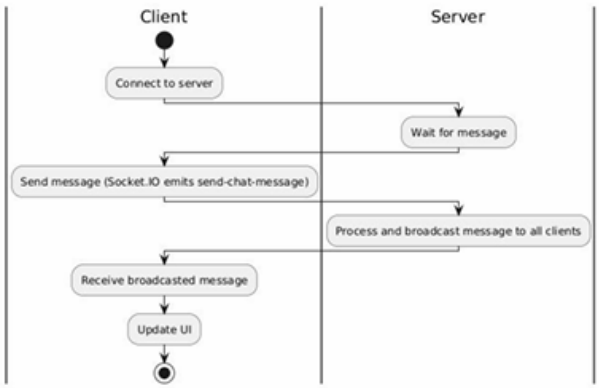


Figure 3.4: Client-server architecture of the proposed chat application.

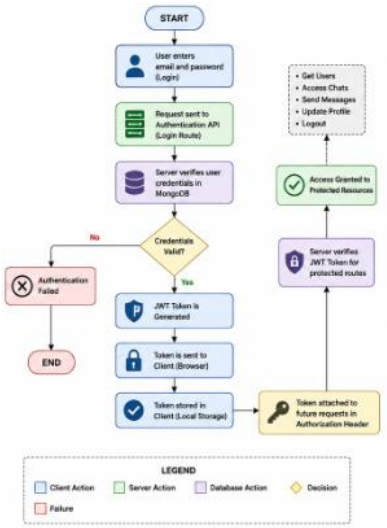


Figure 3.5: Authentication workflow of the proposed system.

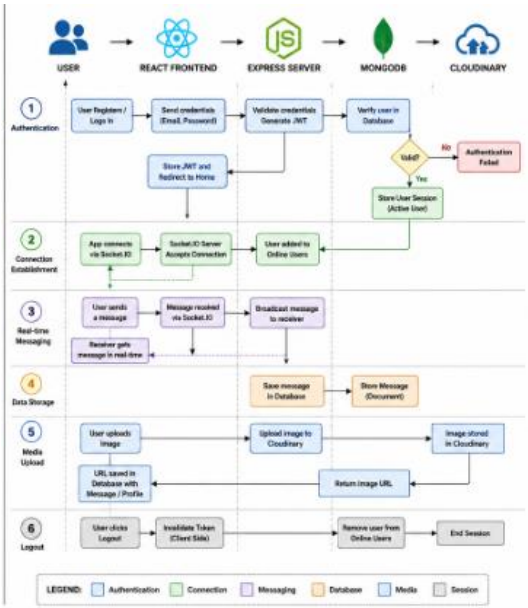


Figure 3.6: High-level workflow of the proposed real-time chat application.

3.3 Database and Module Design

The proposed real-time web-based chat application is designed using a modular architecture with a well-structured database to ensure efficient data management and maintainability. The database stores user information, chat messages, and profile details, while the application modules handle authentication, messaging, profile management, and media upload independently. This modular approach improves code organization, simplifies debugging, and allows future enhancements to be incorporated with minimal modifications.

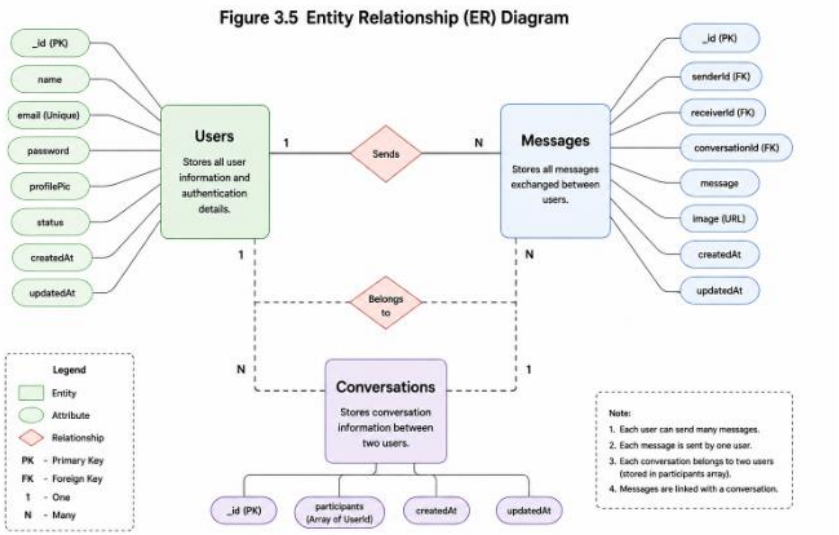


Figure 3.5: Entity Relationship (ER) diagram of the proposed chat application.

Collection	Description
Users	Stores user account details, authentication data, and profile information.
Messages	Stores text messages, media URLs, sender ID, receiver ID, and timestamps.

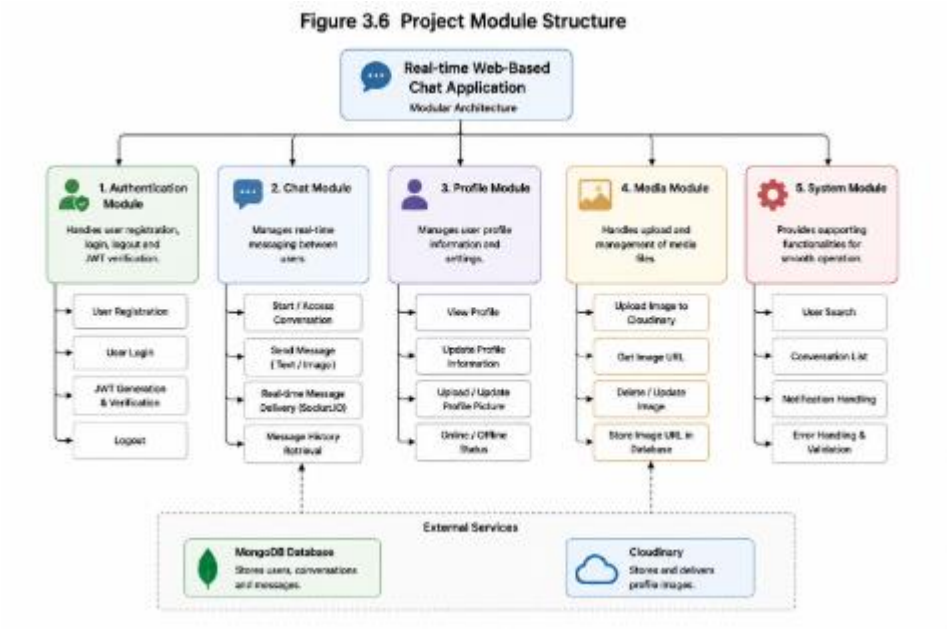
Table 3.6 Database Collections

Explanation

The application primarily uses two collections to manage its data. The **Users** collection contains account and profile information, while the **Messages** collection maintains chat records exchanged between users. This separation improves data organization and simplifies query operations.

Module	Responsibility
Authentication	User registration, login, logout, and JWT verification
Chat	Sending, receiving, and storing messages
Profile	Updating user information and profile picture
Media	Uploading and managing profile images

Figure 3.6 Project Module Structure



Explanation

Figure 3.6 presents the major functional modules of the application. The Authentication module manages user registration and login, the Chat module handles real-time message exchange, the Profile module manages user information and profile images, and the Media module integrates with Cloudinary for image storage. Each module performs a dedicated responsibility while interacting with the backend through well-defined interfaces.

Section Summary

The database and module design provide the structural foundation of the proposed chat application. The use of separate collections for users and messages, together with modular application components, ensures efficient data management, easier maintenance, and improved scalability.

3.4 Chapter Summary

This chapter presented the system analysis and design of the proposed real-time web-based chat application. It discussed the system requirements, overall architecture, database design, and the major functional modules that form the foundation of the application. The analysis demonstrates that the modular client-server architecture, combined with an organized database structure, provides a secure, scalable, and efficient environment for real-time communication. The next chapter focuses on the implementation of the proposed system, describing how each component was developed and integrated to achieve the desired functionality.

CHAPTER 4 : SYSTEM IMPLEMENTATION

4.1 Development Environment

The proposed real-time web-based chat application was developed using a modern full-stack web development environment based on the MERN Stack. The development environment was carefully selected to provide efficient application development, secure user authentication, real-time communication, and simplified deployment. The project follows a modular folder structure, separating the frontend and backend into independent modules, thereby improving maintainability and scalability.

The frontend was developed using React.js with Vite as the build tool, while the backend was implemented using Node.js and Express.js. MongoDB serves as the primary database for storing user and message information, and Cloudinary is integrated for cloud-based image storage. Socket.IO enables persistent bidirectional communication, allowing users to exchange messages instantly.

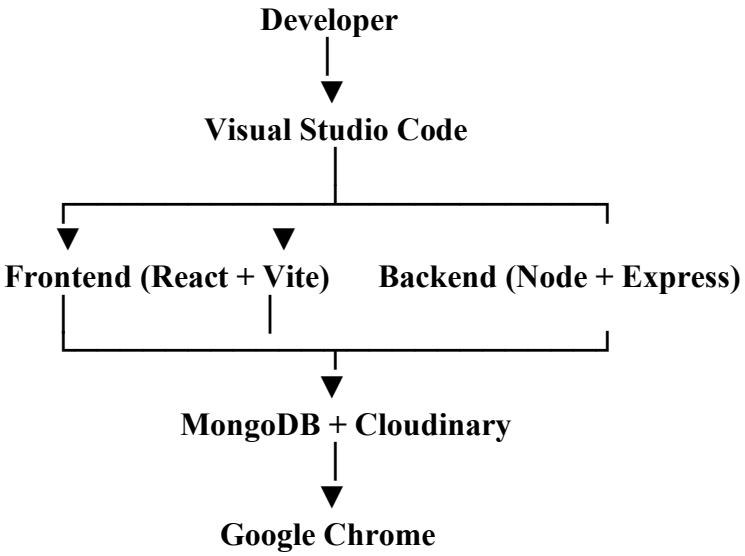


Figure 4.1: Development environment used for implementing the proposed chat application.

Explanation

Figure 4.1 illustrates the development environment adopted for the project. Visual Studio Code was used as the primary development environment. The frontend and backend were developed independently and communicated through REST APIs and Socket.IO. MongoDB was used for persistent data storage, while Cloudinary managed profile image uploads. Google Chrome was used for testing and debugging the application during development.

Tool / Technology	Purpose
Visual Studio Code	Source code development
React.js + Vite	Frontend development
Node.js	JavaScript runtime
Express.js	Backend framework
MongoDB Atlas	Database management
Socket.IO	Real-time communication
Cloudinary	Image storage
Git & GitHub	Version control
Google Chrome	Application testing

Table 4.1 Development Tools

Explanation

Table 4.1 lists the primary development tools used during implementation. Each tool was selected according to its role in the project, enabling efficient development, testing, real-time communication, and secure data management.

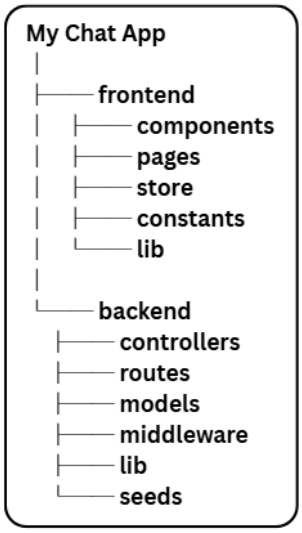


Figure 4.2: High-level directory structure of the developed application.

Section Summary

The development environment combines modern web technologies and development tools to support efficient implementation of the proposed chat application. The modular project structure, along with separate frontend and backend components, simplifies development, testing, and future maintenance while ensuring a scalable foundation for real-time communication.

4.2 Frontend Implementation

The frontend of the proposed chat application was developed using **React.js** with **Vite** as the build tool. The user interface follows a component-based architecture, allowing reusable components to be organized into separate modules. The application provides a responsive interface for user authentication, profile management, and real-time messaging while maintaining a consistent user experience across different screen sizes.

React Router is used for client-side navigation between different pages, while Zustand manages the global application state. Tailwind CSS and DaisyUI are used to design the user interface, providing a modern and responsive layout with minimal custom styling.

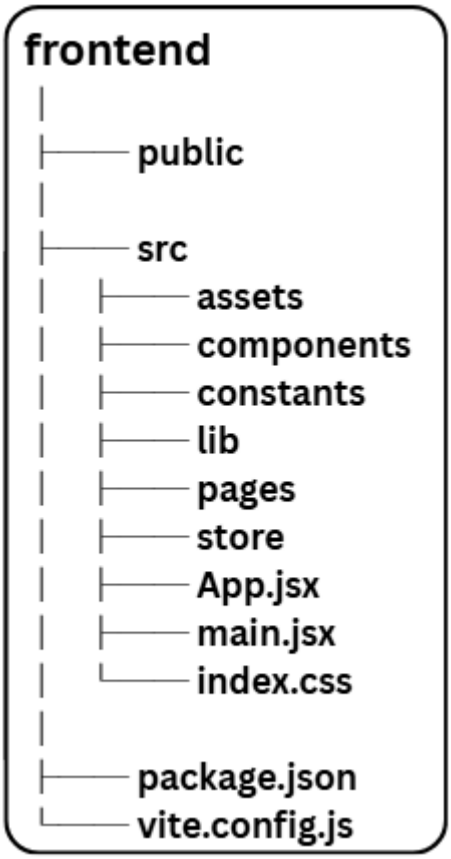


Figure 4.3: Directory structure of the frontend application.

Technology	Purpose
React.js	User Interface Development
Vite	Development and Build Tool
React Router	Client-side Routing
Zustand	Global State Management
Tailwind CSS	Styling Framework
DaisyUI	Pre-built UI Components

Table 4.2 Frontend Technologies

4.2.1 User Interface

The frontend provides separate interfaces for user registration, login, profile management, and chat functionality. Each page is implemented as an independent React component, enabling code reusability and easier maintenance. Navigation between these pages is managed through React Router, ensuring a seamless user experience without full page reloads.

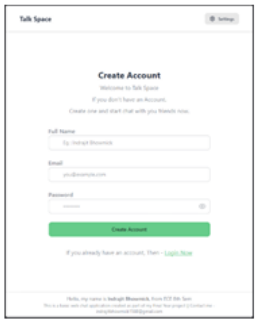


Figure 4.4: User registration page.

Explanation

Figure 4.3 presents the organization of the frontend source code. The components directory contains reusable UI components, pages contains individual application screens, store manages global application state using Zustand, lib stores utility functions, and constants contains application-wide constant values. This modular organization improves code readability and maintainability.

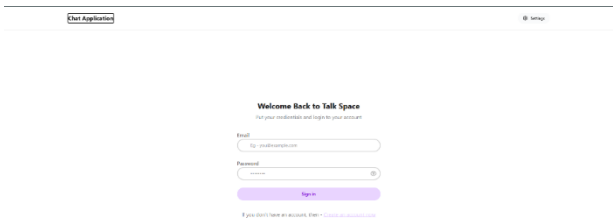


Figure 4.5: User login page.

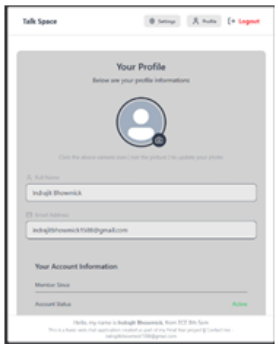


Figure 4.6: Main chat interface.

Figure 4.7: Profile management page.

4.3 Backend Implementation

The backend of the proposed chat application is responsible for processing client requests, managing user authentication, handling real-time communication, and interacting with the database. It is developed using **Node.js** and **Express.js**, following a modular architecture in which routes, controllers, models, and middleware are organized into separate directories. This organization improves code readability, simplifies maintenance, and supports future feature expansion.
The backend exposes RESTful APIs for user registration, login, profile management, and message retrieval. In addition, Socket.IO is integrated with the server to enable real-time communication between connected users.

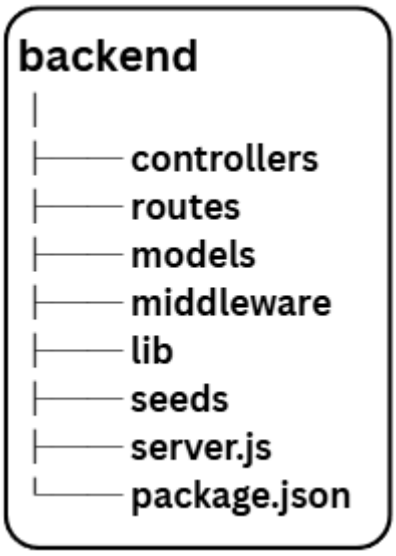


Figure 4.8: Directory structure of the backend application.

Module	Responsibility
Controllers	Process incoming requests and execute business logic
Routes	Define API endpoints
Models	Define MongoDB document schemas
Middleware	Handle authentication and request validation
Lib	Database connection and utility functions
Seeds	Populate the database with sample data

Table 4.3 Backend Modules

Explanation

Figure 4.8 shows the organization of the backend source code. The project is divided into controllers, routes, models, middleware, and utility files, ensuring that each component performs a dedicated responsibility. This modular structure improves maintainability and simplifies application development.

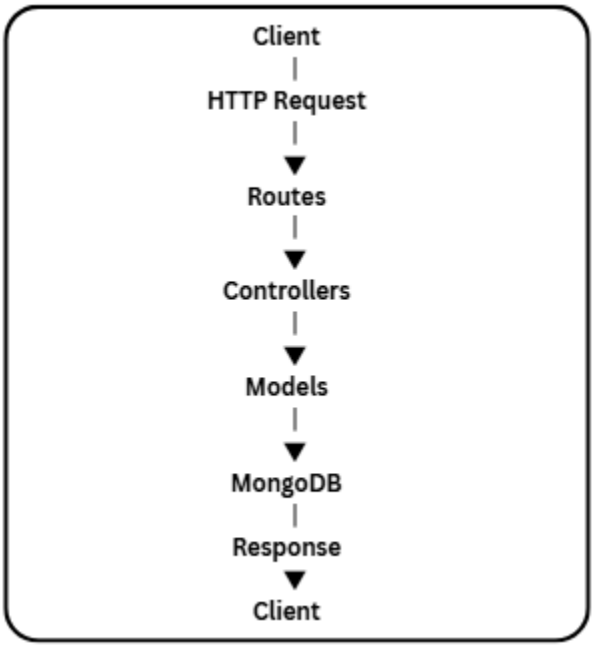


Figure 4.9: Flow of a client request through the backend.

Explanation

Figure 4.9 illustrates how the backend processes client requests. Incoming requests are first received by the defined routes, which forward them to the appropriate controllers. The controllers execute the required business logic, interact with the database through the models, and return the response to the client.

Section Summary

The backend implementation follows a modular architecture that separates routing, business logic, database interaction, and middleware into dedicated modules. This organization improves code maintainability while supporting secure authentication, efficient data processing, and seamless integration with the real-time communication system.

4.4 Database and Authentication

The proposed chat application utilizes MongoDB to store user information and chat messages. The database is organized into separate collections, allowing efficient management of user accounts and conversations. Authentication is implemented using JSON Web Tokens (JWT), ensuring that only authorized users can access protected resources. Passwords are encrypted using **bcrypt** before being stored in the database, enhancing the security of user credentials.

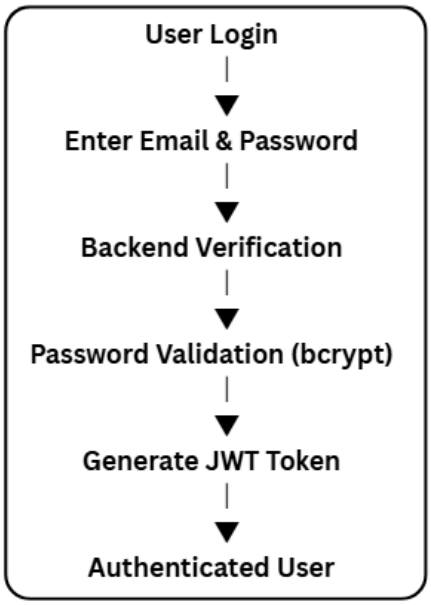


Figure 4.10: Authentication workflow of the proposed chat application.

Explanation

Figure 4.10 illustrates the authentication process. After the user submits valid login credentials, the backend verifies the password using bcrypt. If the credentials are valid, a JWT token is generated and returned to the client. The token is then used to authorize access to protected resources throughout the user's session.

Field	Description
_id	Unique user identifier
fullName	User's full name
email	Registered email address
password	Encrypted password
profilePic	Profile image URL
createdAt	Account creation timestamp

Table 4.4 User Collection

Explanation

The **Users** collection stores account and profile information for each registered user. Passwords are stored in encrypted form, while profile images are referenced using Cloudinary URLs.

Field	Description
_id	Unique message identifier
senderId	Sender's user ID
receiverId	Receiver's user ID
text	Message content
image	Image URL (if any)
createdAt	Message timestamp

Table 4.5 Message Collection

Explanation

The **Messages** collection stores the conversation history between users. Each document contains the sender, receiver, message content, optional image URL, and timestamp, allowing efficient retrieval of chat history.

Section Summary

The application implements a secure authentication mechanism using JWT and bcrypt while organizing user and message information into separate MongoDB collections. This design ensures secure access control, efficient data management, and reliable storage of conversations.

4.5 Real-Time Communication

Real-time communication is the primary functionality of the proposed chat application. This feature is implemented using **Socket.IO**, which establishes a persistent bidirectional connection between the client and the server. Unlike the traditional HTTP request-response model, Socket.IO allows messages to be transmitted instantly without requiring repeated page refreshes or polling requests.

When a user connects to the application, a socket connection is established with the server. The server maintains a list of active users and continuously listens for communication events. Whenever a message is sent, it is immediately delivered to the intended recipient while simultaneously being stored in the database for future retrieval.

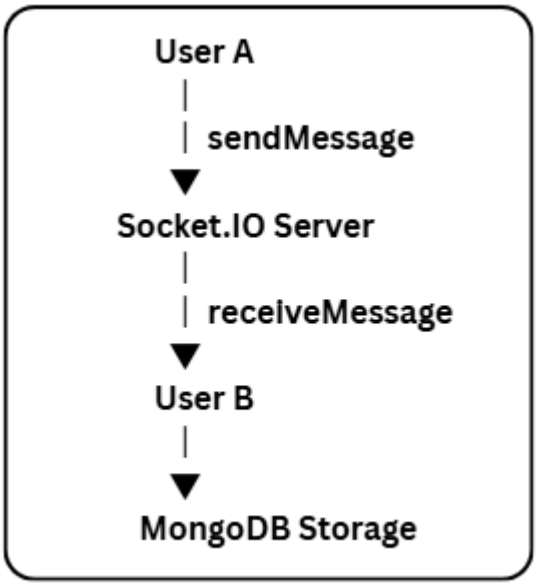


Figure 4.11: Real-time message communication using Socket.IO.

Explanation

Figure 4.11 illustrates the message transmission process. When User A sends a message, the Socket.IO server receives the event, forwards it instantly to User B if online, and stores the message in MongoDB. This process enables low-latency communication while preserving the conversation history.

Event	Description
Connection	Establishes a socket connection between the client and server.
sendMessage	Sends a message from the sender to the server.
receiveMessage	Delivers the message to the intended recipient.
Disconnect	Removes the user from the active connection list when they leave the application.

Table 4.6 Socket Events

Explanation

The application uses Socket.IO events to manage real-time communication. These events enable instant message delivery, user connection management, and synchronization between multiple connected clients.

Figure 4.12: Workflow of message processing in the proposed chat application.

Explanation

Figure 4.12 shows the sequence of operations performed when a user sends a message. The message is received by the server, stored in the database, delivered to the recipient in real time, and immediately displayed on the chat interface.

Section Summary

The real-time communication module enables instant and reliable message exchange through Socket.IO. Persistent socket connections, event-driven communication, and database integration ensure efficient message delivery while maintaining complete conversation history.

4.6 Testing and System Outputs

The developed chat application was tested to verify its functionality, usability, and real-time communication capabilities. Each major feature was evaluated individually to ensure that the application performs as expected under normal operating conditions. Screenshots of the implemented interfaces are presented below to demonstrate the successful execution of the developed modules.

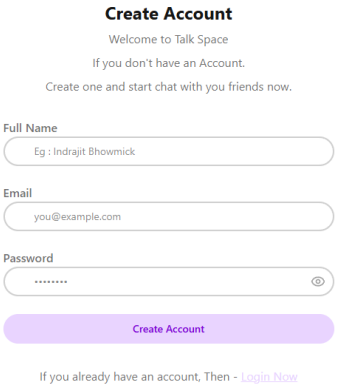


Figure 4.13: User registration interface.

Explanation

The registration page allows new users to create an account by providing the required information. After successful validation, the user details are securely stored in the database.

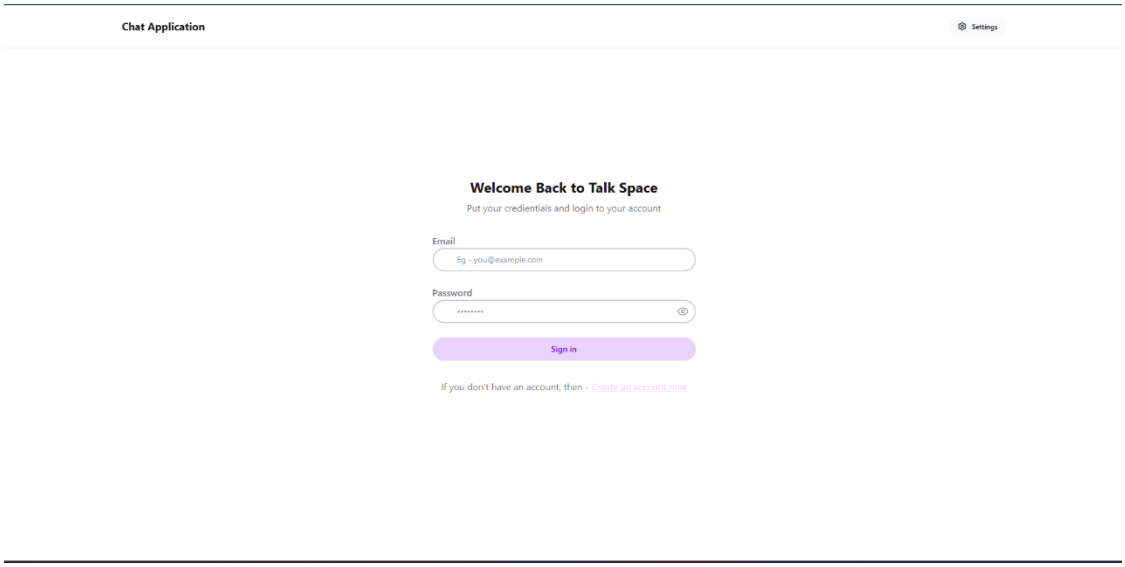


Figure 4.14: User login interface.

Explanation

The login page authenticates registered users using their email address and password. Upon successful authentication, users are redirected to the main chat dashboard.

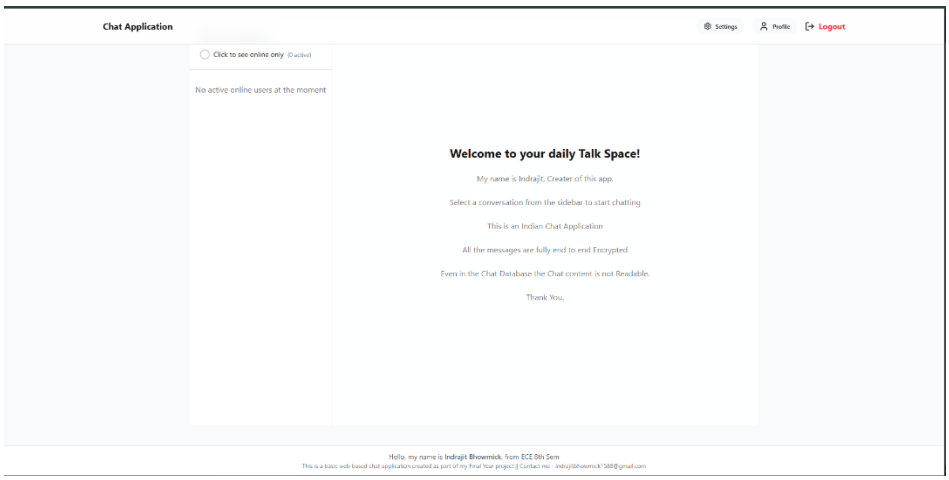


Figure 4.15: Main chat dashboard.

Explanation

The chat dashboard displays available users, conversation history, and the messaging interface. Messages are exchanged instantly through Socket.IO without refreshing the page.

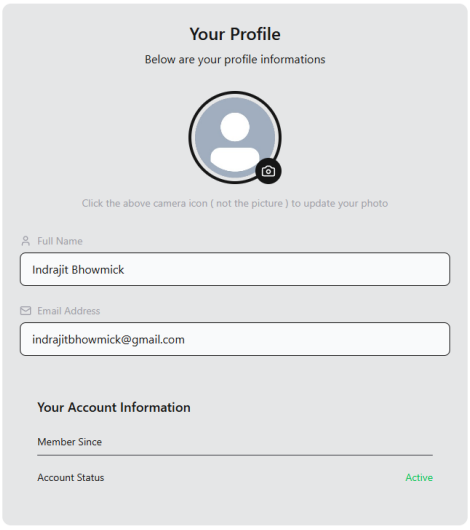


Figure 4.16: Profile management interface.

Explanation

The profile page allows users to update their personal information and profile picture. Uploaded images are stored in Cloudinary and linked to the user's account.

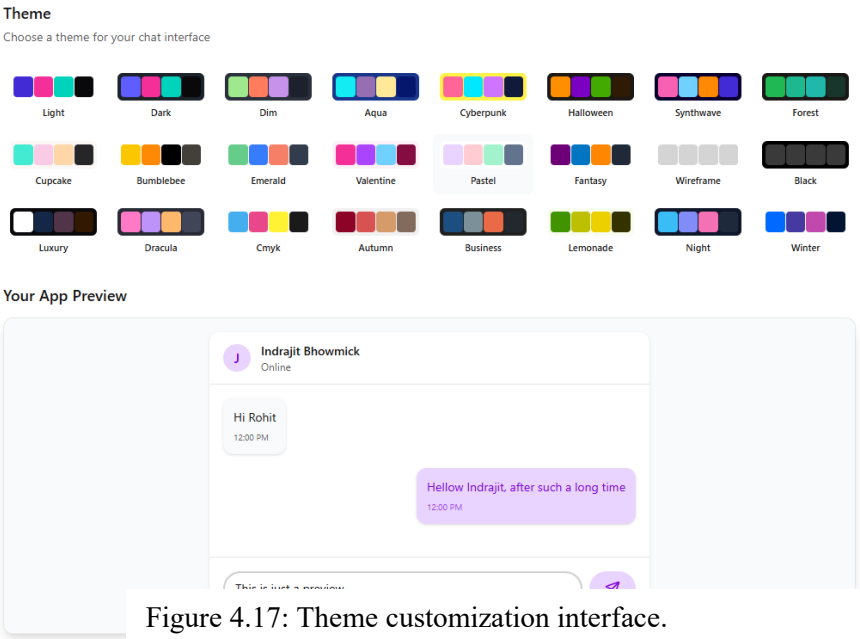


Figure 4.17: Theme customization interface.

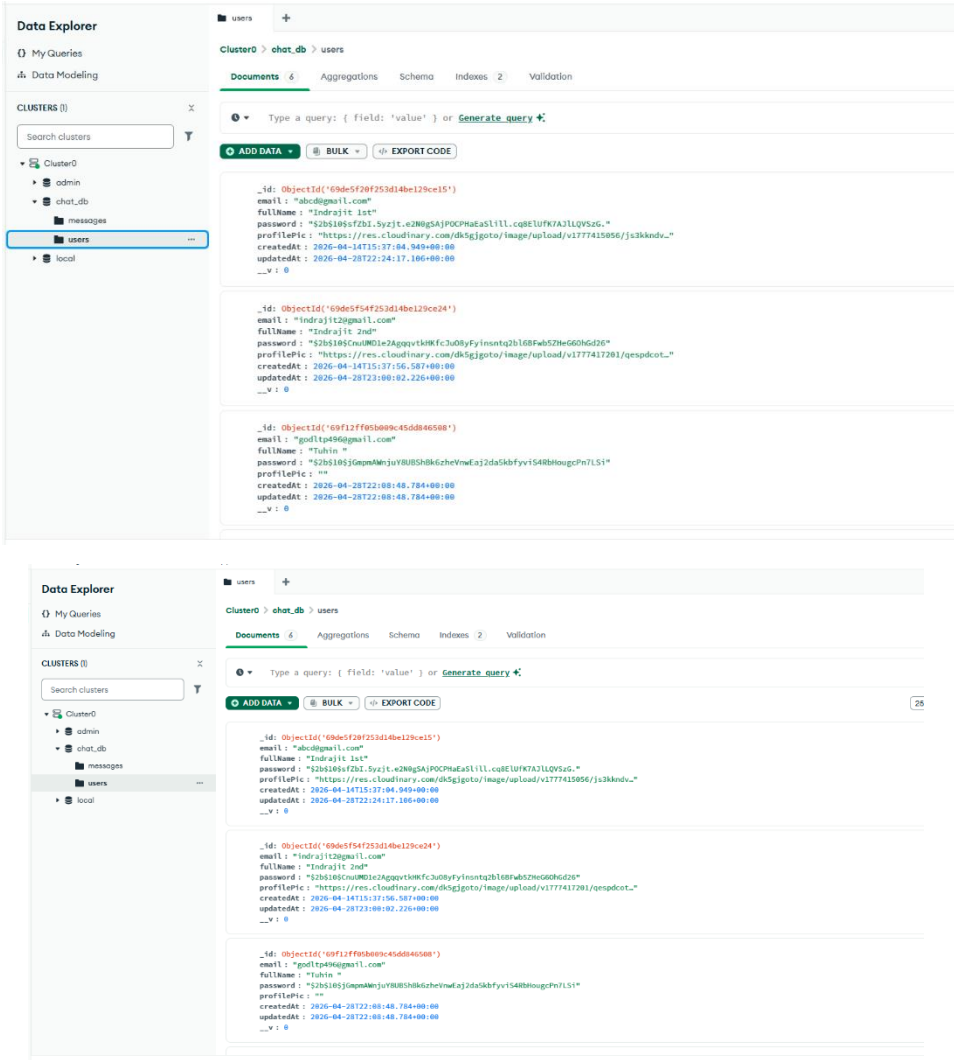


Figure 4.18: MongoDB collections.

4.7 Chapter Summary

This chapter described the implementation of the proposed real-time web-based chat application. It presented the development environment, frontend and backend implementation, database design, authentication mechanism, real-time communication process, and the testing results. The successful integration of these components demonstrates the effectiveness of the MERN Stack and Socket.IO in developing a secure, scalable, and responsive real-time messaging platform.

Test Case	Expected Result	Status
User Registration	Account created successfully	Passed
User Login	Authentication successful	Passed
Send Message	Message delivered instantly	Passed
Receive Message	Message displayed immediately	Passed
Profile Update	Information updated successfully	Passed
Image Upload	Image uploaded successfully	Passed
Logout	Session terminated securely	Passed

Table 4.7 Functional Testing

Explanation

Table 4.7 summarizes the functional testing performed on the developed application. All major functionalities were successfully tested and produced the expected results.

CHAPTER 5 : RESULTS AND DISCUSSION

5.1 Results

The proposed real-time web-based chat application was successfully developed and tested using the MERN Stack and Socket.IO. The implemented system fulfills the primary objectives defined during the project planning stage by providing secure user authentication, instant messaging, profile management, and real-time communication through a responsive web interface.

The application was evaluated by testing its major functionalities under different user scenarios. The results confirmed that the system performs reliably, allowing users to register, log in securely, exchange messages instantly, update profile information, and maintain conversation history. The integration of Socket.IO enables low-latency communication, while MongoDB efficiently stores user and message data. Overall, the application demonstrated stable performance and a user-friendly interface throughout the testing process.

Briefly discuss the outcomes of your project:

- Successful user registration and login.
- Secure authentication using JWT.
- Instant message delivery using Socket.IO.
- Profile image upload through Cloudinary.
- Real-time online user status.
- Responsive interface across different screen sizes.

Feature	Implementation Status
User Registration	✓ Implemented
User Authentication	✓ Implemented
Real-Time Messaging	✓ Implemented
Online User Status	✓ Implemented
Profile Management	✓ Implemented
Profile Image Upload	✓ Implemented
Responsive User Interface	✓ Implemented

Table 5.1 Feature Implementation Status

Explanation

Table 5.1 summarizes the major features implemented in the proposed system. All planned core functionalities were successfully completed and verified during testing.

5.2 Performance Analysis

The performance of the application was evaluated based on responsiveness, real-time message delivery, database operations, and overall usability. The use of Socket.IO enabled near-instant message transmission between connected users without requiring page refreshes. MongoDB efficiently handled user and conversation data, while the React frontend provided smooth navigation and responsive interaction.

During testing, the application maintained stable communication and accurately synchronized chat messages between users. Authentication requests were processed successfully, and profile updates were reflected immediately after submission. The modular architecture also contributed to easier debugging and maintenance.

Parameter	Observation
Authentication	Successful
Message Delivery	Instant
Database Response	Efficient
User Interface	Responsive
Real-Time Communication	Stable
Overall Performance	Satisfactory

Table 5.2 Performance Evaluation

Discuss:

- Instant message delivery.
- Low communication latency.
- Stable communication through Socket.IO.
- Efficient MongoDB data retrieval.
- Responsive UI performance.

5.3 Advantages and Limitations

The performance of the application was evaluated based on responsiveness, real-time message delivery, database operations, and overall usability. The use of Socket.IO enabled near-instant message transmission between connected users without requiring page refreshes. MongoDB efficiently handled user and conversation data, while the React frontend provided smooth navigation and responsive interaction.

Advantages

- Provides real-time communication using Socket.IO.
- Secure authentication using JWT and bcrypt.
- Responsive and user-friendly interface.
- Modular architecture for easier maintenance.
- Efficient storage using MongoDB.
- Cloud-based profile image management using Cloudinary.

Limitations

- Supports only one-to-one messaging.
- Voice and video calling are not available.
- End-to-end encryption has not been implemented.
- Push notifications are not supported.
- Internet connectivity is required for communication.

5.4 Discussion

The developed application demonstrates the practical implementation of a modern real-time communication platform using the MERN Stack. The integration of frontend, backend, database, and Socket.IO successfully achieves secure authentication and low-latency messaging. The modular system architecture simplifies future enhancements and provides a strong foundation for extending the application with additional communication features.

Although the project focuses on one-to-one messaging, its design allows future integration of group chats, multimedia communication, and AI-powered features without major architectural modifications. The successful implementation confirms the suitability of the chosen technologies for developing scalable and responsive web applications.

5.5 Chapter Summary

This chapter presented the implementation results and performance evaluation of the proposed real-time web-based chat application. The testing confirmed that the system successfully fulfills its intended objectives by providing secure authentication, efficient data management, and reliable real-time communication. The observed advantages and identified limitations provide valuable insights for future development and enhancement of the application.

CHAPTER 6 : CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The primary objective of this project was to design and implement a secure, scalable, and responsive real-time web-based chat application using the MERN Stack and Socket.IO. The developed system successfully enables instant communication between users through persistent bidirectional connections while providing secure user authentication and efficient data management.

The application integrates modern web technologies, including React.js for the frontend, Node.js and Express.js for the backend, MongoDB for data storage, and Socket.IO for real-time communication. Additional features such as profile management, image upload, and online user status further enhance the usability of the system. The modular architecture adopted during development improves maintainability and provides flexibility for future enhancements.

The implementation and testing results demonstrate that the proposed system satisfies the objectives defined at the beginning of the project. Overall, the application provides a practical demonstration of modern full-stack web development and real-time communication techniques, making it suitable for both academic learning and future development.

6.2 Future Scope

Although the developed application successfully implements the core functionalities of a real-time messaging platform, several additional features can be incorporated in future versions to improve its functionality and user experience. Potential enhancements include:

- Group chat functionality.
- Voice and video calling.
- End-to-end message encryption.
- Push notifications for offline users.
- File and document sharing.
- Message search and filtering.
- AI-powered chatbot integration.
- Message translation and sentiment analysis.
- Read receipts and typing indicators.
- Deployment using cloud platforms with CI/CD support.

These enhancements would further improve the scalability, security, and usability of the application while expanding its practical applicability.

6.3 Chapter Summary

This chapter presented the conclusion and future scope of the proposed real-time web-based chat application. The successful implementation of the system demonstrates the effectiveness of the MERN Stack and Socket.IO in developing secure and responsive communication platforms. The proposed application provides a strong foundation for future enhancements and can be extended to support advanced communication features and intelligent services.

REFERENCES

[1] Agarwal, R., Gao, G., DesRoches, C., et al.: The digital transformation of healthcare: current status and the road ahead. Inf. Syst. Res. 21, 796–809 (2010).

[2] Aluttis, C., Bishaw, T., Frank, M.W.: The workforce for health in a globalized context – global shortages and international migration. Glob. Health Action 7 (2014). https://www. ncbi.nlm.nih.gov/pmc/articles/PMC3926986/.

[3] Bergkvist, L., Rossiter, J.R.: The predictive validity of multiple-item versus single-item measures of the same constructs. J. Mark. Res. 44, 175–184 (2007).
[4] Chaffey, D.: Mobile Marketing Statistics http://www.smartinsights.com/ . Accessed Feb 2017.

[5] Filler, A., Kowatsch, T., Haug, S., et al.: MobileCoach: a novel open source platform for the design of evidence-based, scalable and low-cost behavioral health interventions. In: 14th Annual Wireless Telecommunications Symposium (WTS 2015), New York (2015).

[6] Flückiger, C., Del Re, A.C., Wampold, B.E., et al.: How central is the alliance in psychotherapy? A multilevel longitudinal meta-analysis. J. Couns. Psychol. 59, 10–17 (2012).

[7] Haug, S., Kowatsch, T., Paz Castro, R., et al.: Efficacy of a web- and text messaging based intervention to reduce problem drinking in young people: study protocol of a cluster-randomised controlled trial. BMC Public Health 14, 1–8 (2014).

[8] Haug, S., Paz Castro, R., Filler, A., et al.: Efficacy of an internet and SMS-based integrated smoking cessation and alcohol intervention for smoking cessation in young people: study protocol of a two-arm cluster randomised controlled trial. BMC Public Health 14, 1140 (2014).

[9] Haug, S., Paz, R., Kowatsch, T., et al.: Efficacy of a web- and text messaging-based intervention to reduce problem drinking in adolescents: Results of a cluster-randomised controlled trial. J. Consult. Clin. Psychol. 85, 147–159 (2017).

[10] Kamis, A., Koufaris, M., Stern, T.: Using an attribute-based decision support system for user-customized products online: an experimental investigation. MIS Q. 32, 159–177 (2008).

[11] Lilian, S., Gregor, W., Sarah, G., et al.: MIKE- Medien, Interaktion, Kinder, Eltern. Zürcher Hochschule für Angewandte Wissenschaften, Zürich (2015).

[12] Nielsen, J., Landauer, T.K.: A mathematical model of the finding of

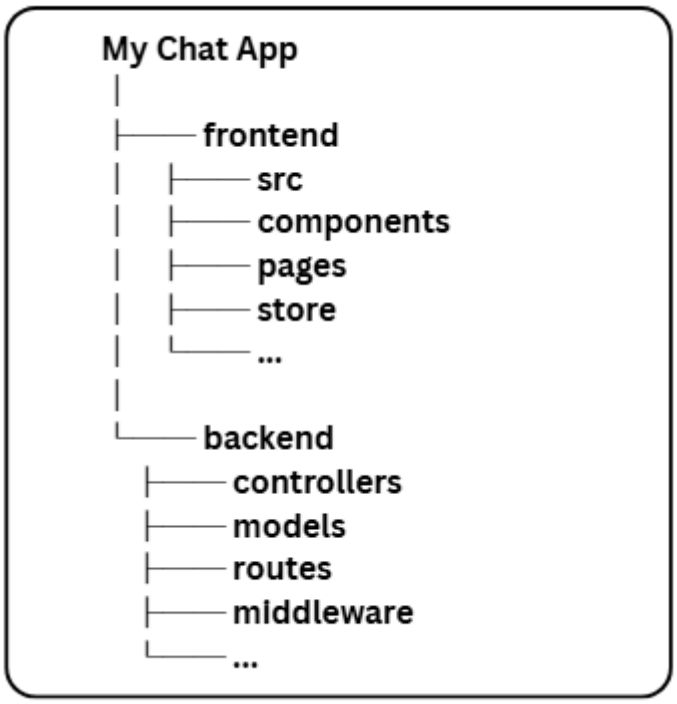
usability problems. In: CHI 1993 Proceedings of the INTERACT 19
[13] Tams, S., Hill, K., de Guinea, A.O., et al.: NeuroIS - alternative or complement to existing methods? Illustrating the holistic effects of neuroscience and self-reported data in the context of technostress research. J. Assoc. Inf. Syst. 15, Article 1 (2014).

[14] Amitha, K., Ram Manohar Reddy, M., Yashwanth, K., Shylaja, K., Rahul Reddy, M., Srinu, B., & Dharnasi, P. (2026). AI empowered security monitoring system with the help of deployed ML models. International Journal of Computer Technology and Electronics Communication (IJCTEC), 9(1), 69–73.

[15] Gogada, S., Gopichand, K., Reddy, K. C., Keerthana, G., Nithish Kumar, M., Shivalingam, N., & Dharnasi, P. (2026). Cloud computing/deep learning customer churn prediction for SaaS platforms. International Journal of Computer Technology and Electronics Communication (IJCTEC), 9(1), 74–78..

APPENDIX

A. Project Directory Structure



B. API Endpoints

Method	Endpoint	Description
POST	/api/auth/signup	Register user
POST	/api/auth/login	Login
POST	/api/auth/logout	Logout
GET	/api/messages/:id	Get messages
POST	/api/messages/send/:id	Send message
PUT	/api/profile/update	Update profile

C. Important Code Snippets

```
export const generateToken = (userId, res) => {
   const token = jwt.sign(
      { userId },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
   );

   res.cookie("jwt", token, {
      maxAge: 7 * 24 * 60 * 60 * 1000,
      httpOnly: true,
   });
};
```

JWT Generation

```
io.on("connection", (socket) => {
   console.log(socket.id);
});
```

Socket Connection

```
const userSchema = new mongoose.Schema({
   fullName: String,
   email: String,
   password: String,
});
```

User Schema

D. Installation Guide

1. Clone Repository
2. npm install
3. Configure .env
4. npm run dev
5. Open localhost:5173

E. GitHub Repository

Project Repository:

https://github.com/IndrajitOD/Chat-App-Full-Stack.git